

云计算基础介绍



目录

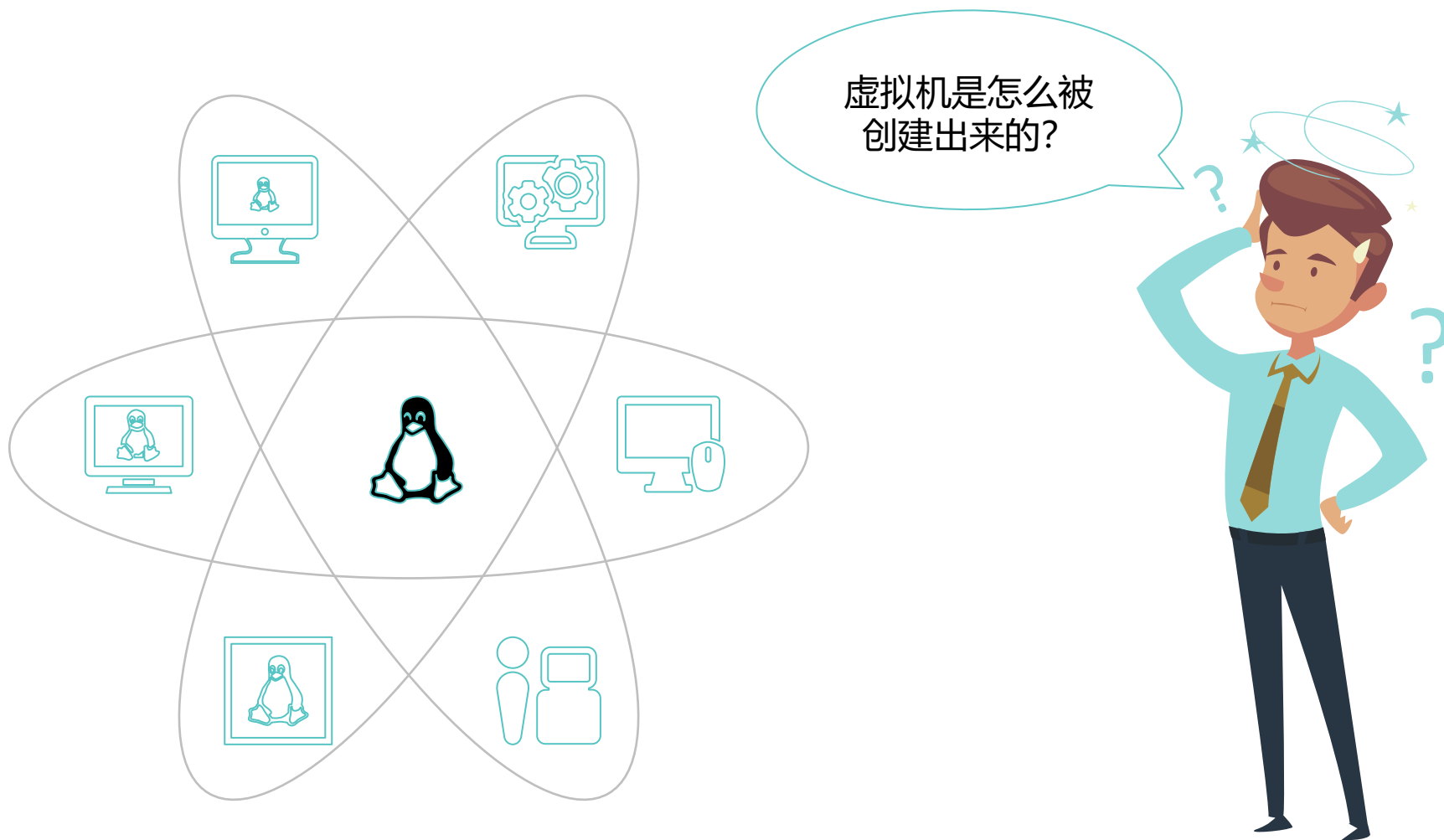
1. 虚拟化技术

- 计算虚拟化
- 网络虚拟化
- 存储虚拟化
- 虚拟化特性

2. OpenStack

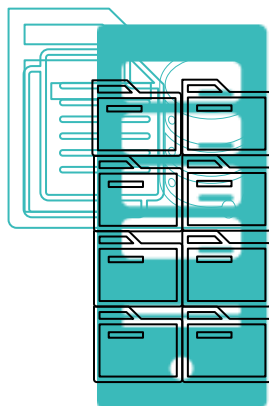
3. 容器技术

虚拟机是什么？

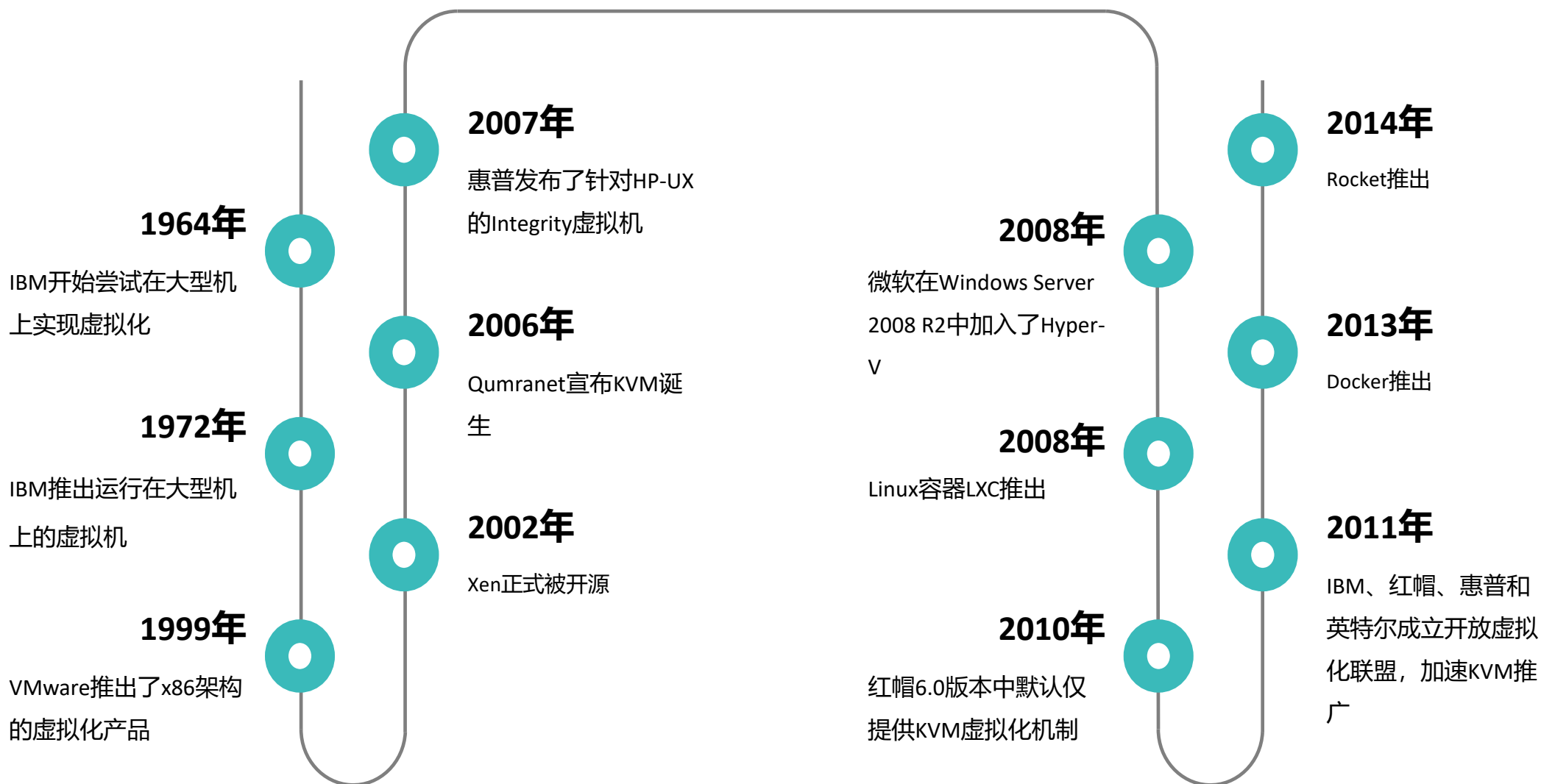


什么是虚拟化

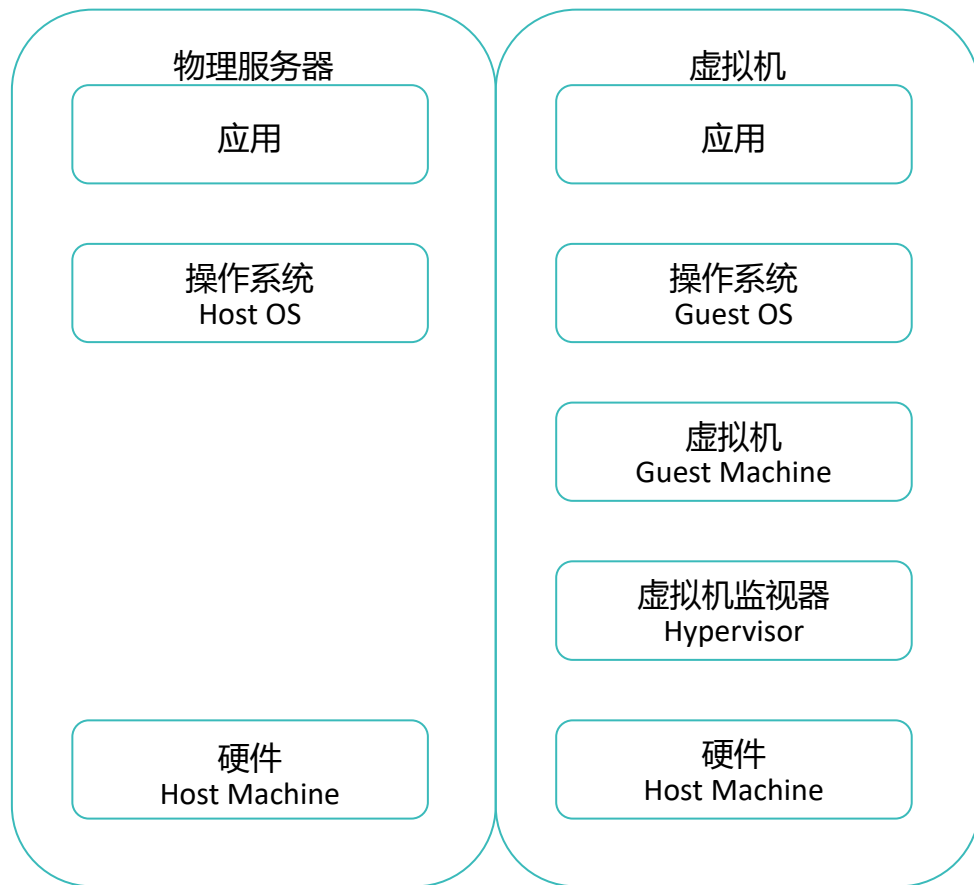
- 虚拟化的本质就是将原先的物理设备进行逻辑化，转化为一个文件夹或文件，实现软硬件的解耦。



计算虚拟化发展史



计算虚拟化中的重要概念



Guest OS:

虚拟机操作系统

Guest Machine:

虚拟出来的虚拟机

Hypervisor:

虚拟化软件层/虚拟机监控机

(Virtual Machine Monitor, VMM)

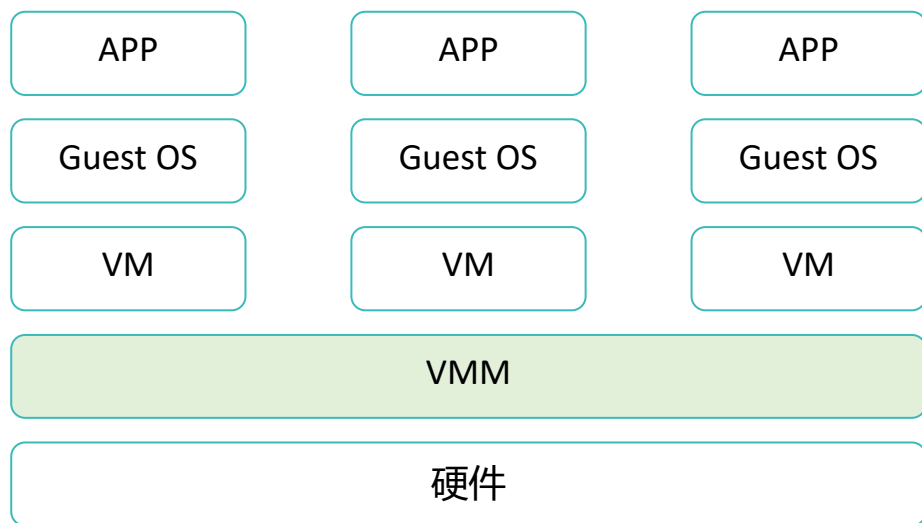
Host OS:

运行在物理机之上的OS

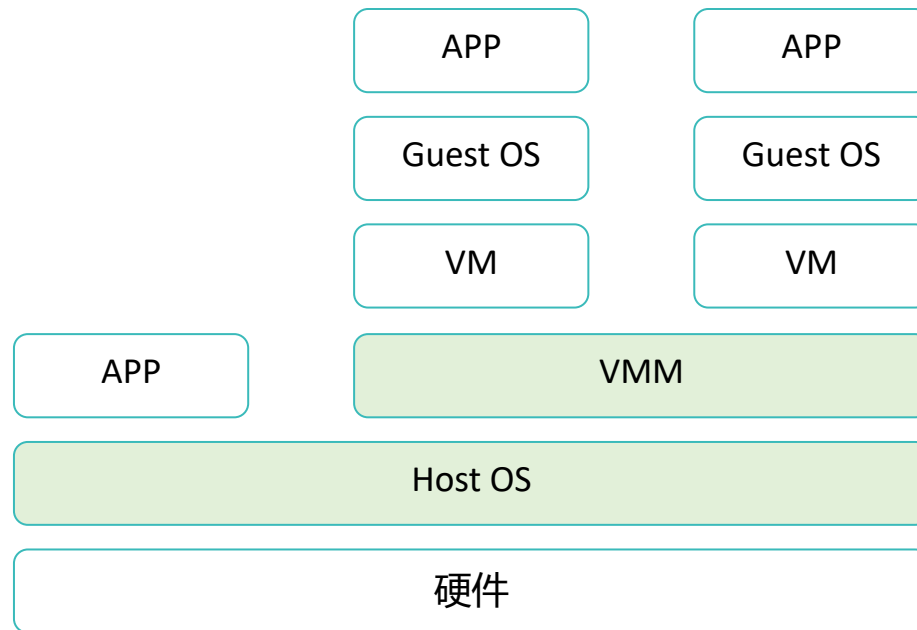
Host Machine:

物理机

计算虚拟化的分类

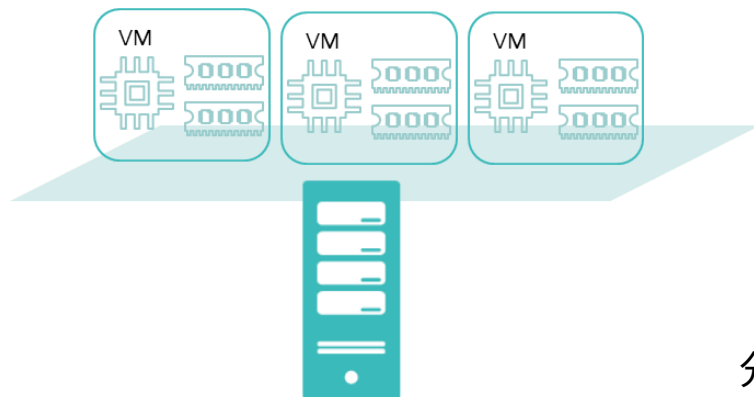


裸金属型虚拟化
(I 型)

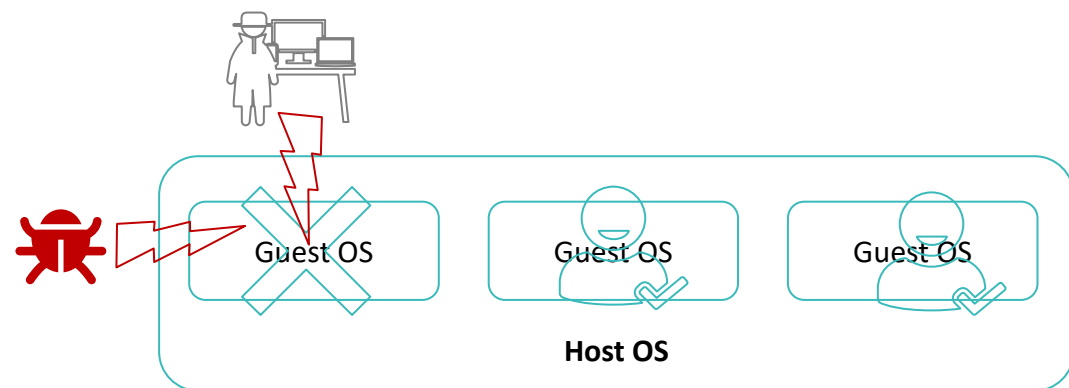


宿主型虚拟化
(II 型)

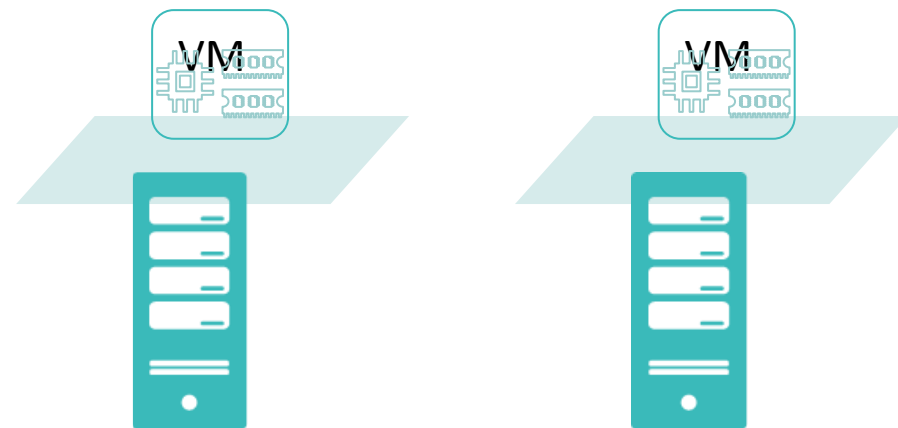
虚拟化的特点



分区
封装

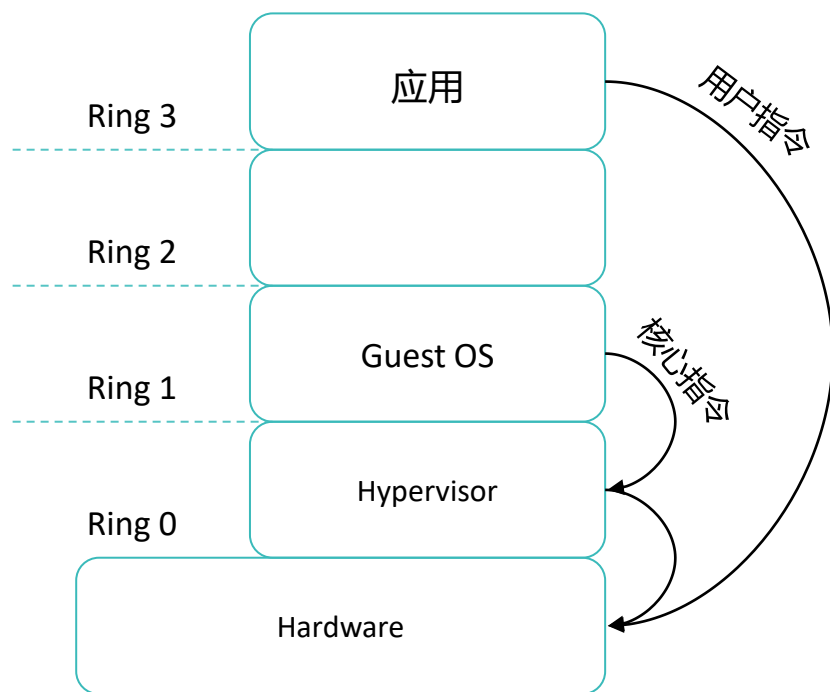


隔离
独立

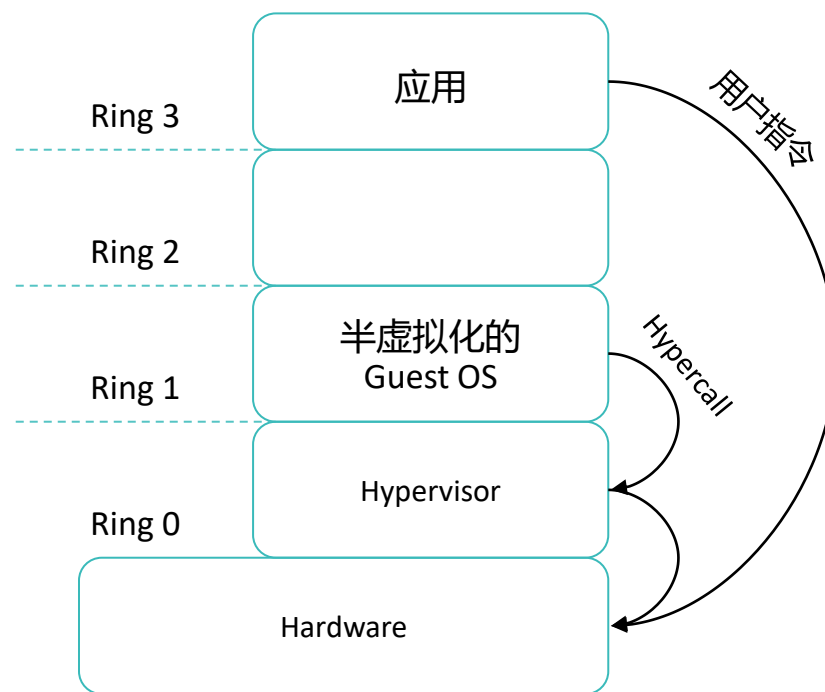


CPU虚拟化 (X86)

全虚拟化

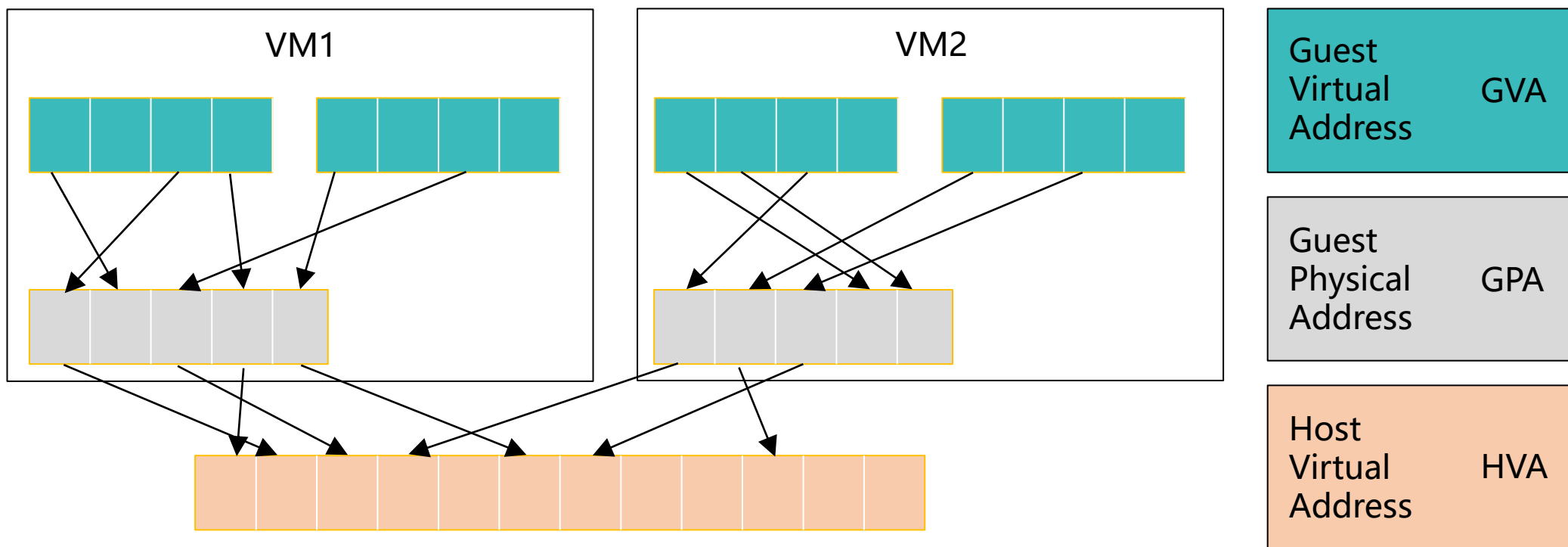


半虚拟化



内存虚拟化

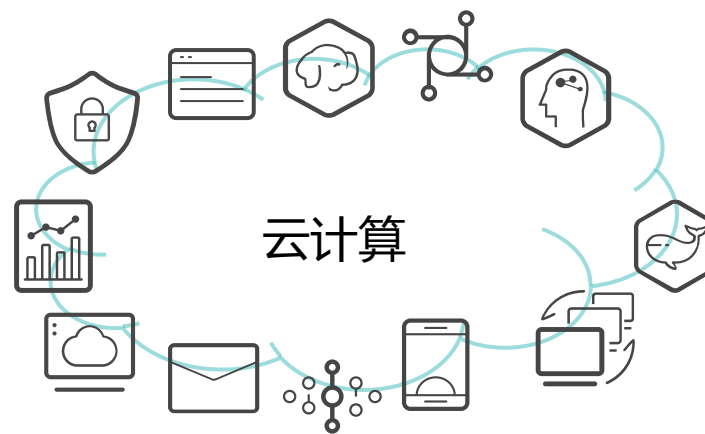
- 内存虚拟化：把物理机的真实物理内存统一管理，包装成多个虚拟机的内存给若干虚拟机使用。KVM 通过内存虚拟化共享物理系统内存，动态分配给虚拟机。



I/O虚拟化

- 模拟(完全虚拟): 完全使用软件来模拟真实硬件, 模拟通常硬件, 例如键盘鼠标, 通过焦点捕获, 焦点被哪个主机捕获就被哪个主机使用, 性能很差。
- 半虚拟化: 对硬件驱动由前端(IO frontend)直接转到后端(IO backend)调用, 通常仅适用于硬盘和网卡, 性能高。
- IO-through: IO透传, 直接分配给虚拟机物理设备, 例如直接分配一个硬盘或网卡给虚拟机, 需要硬件具备IO透传技术, 在Xen下由Dom0分配, 但是访问使用直接使用, 不经过Dom0, 需要硬件支持。

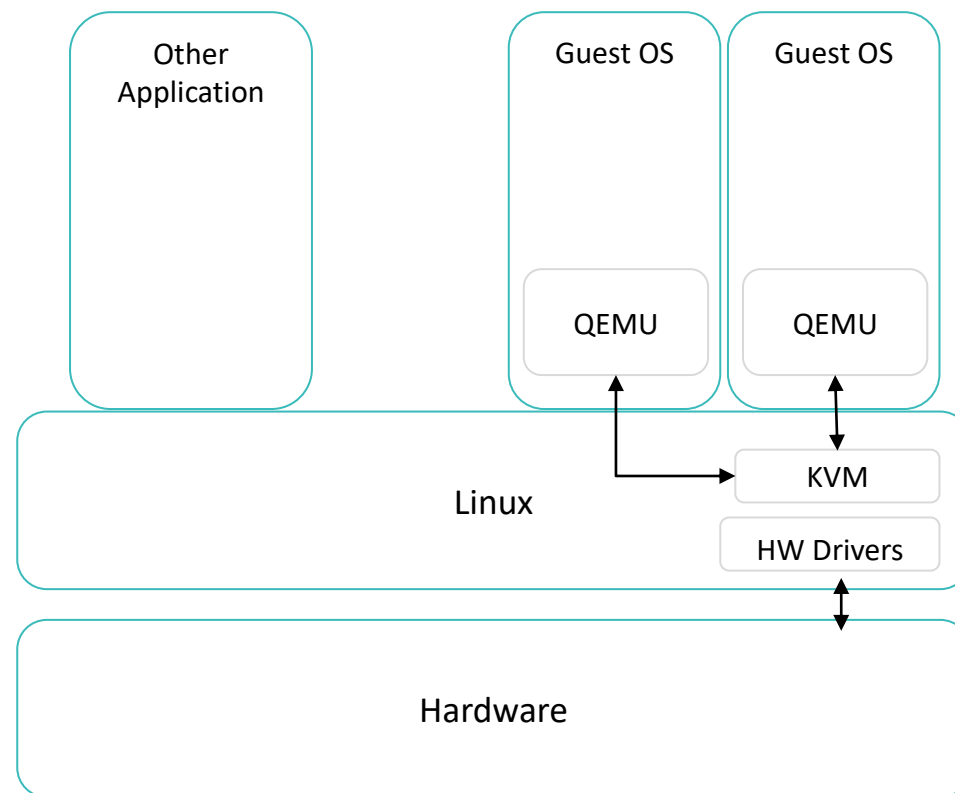
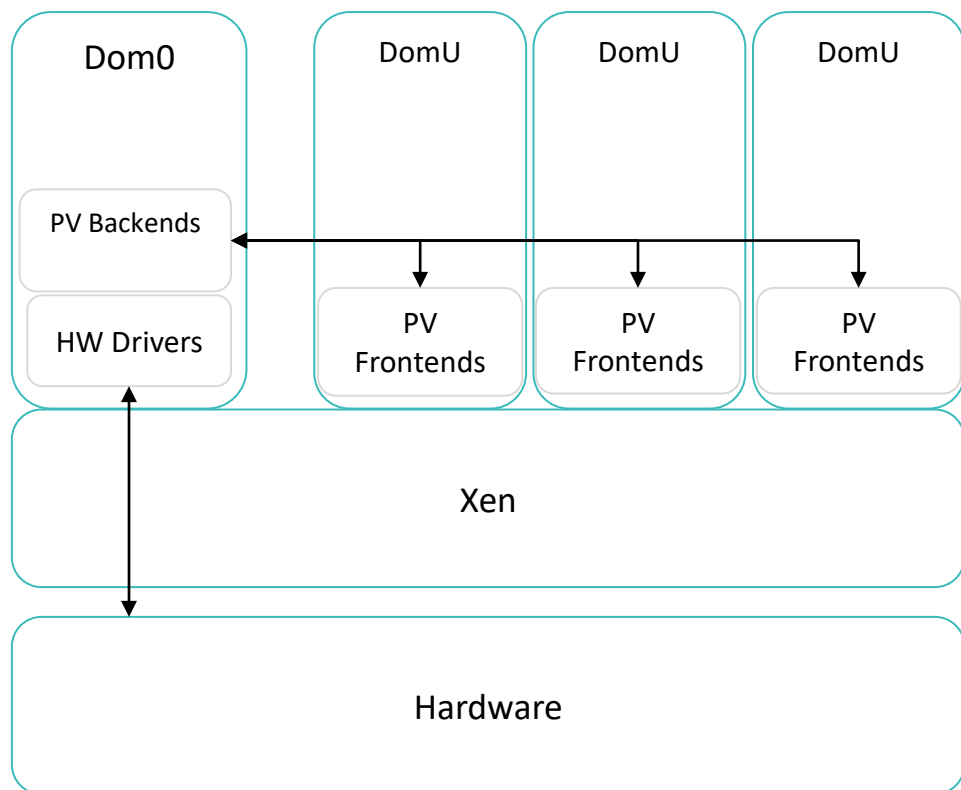
云计算和虚拟化



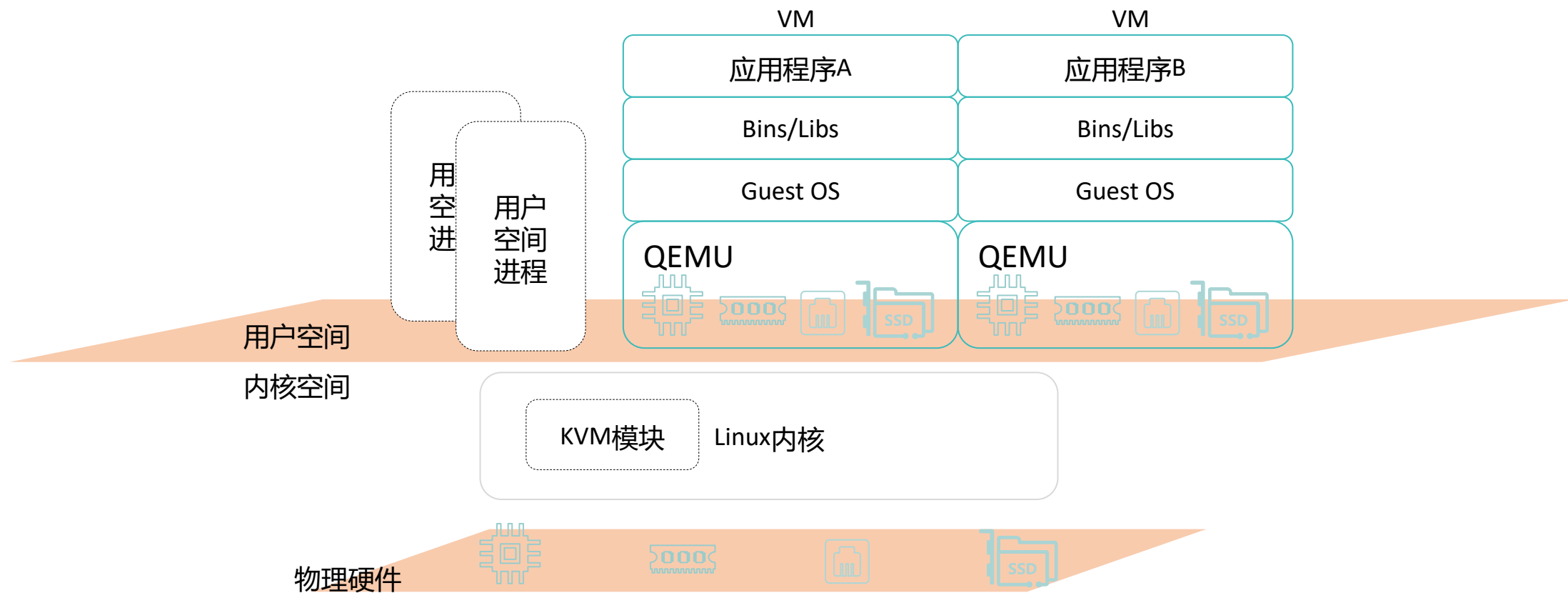
主流计算虚拟化技术



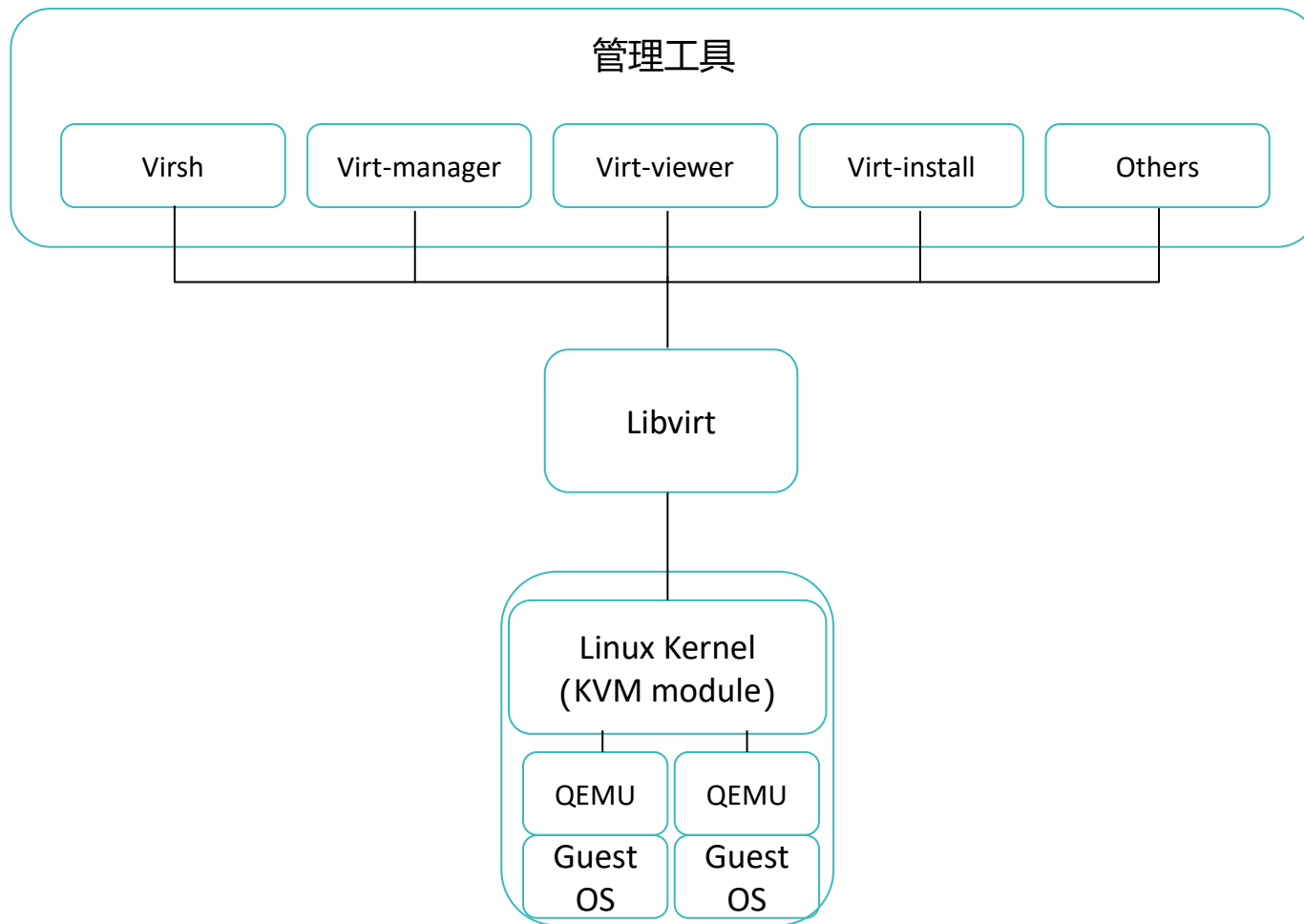
Xen vs KVM



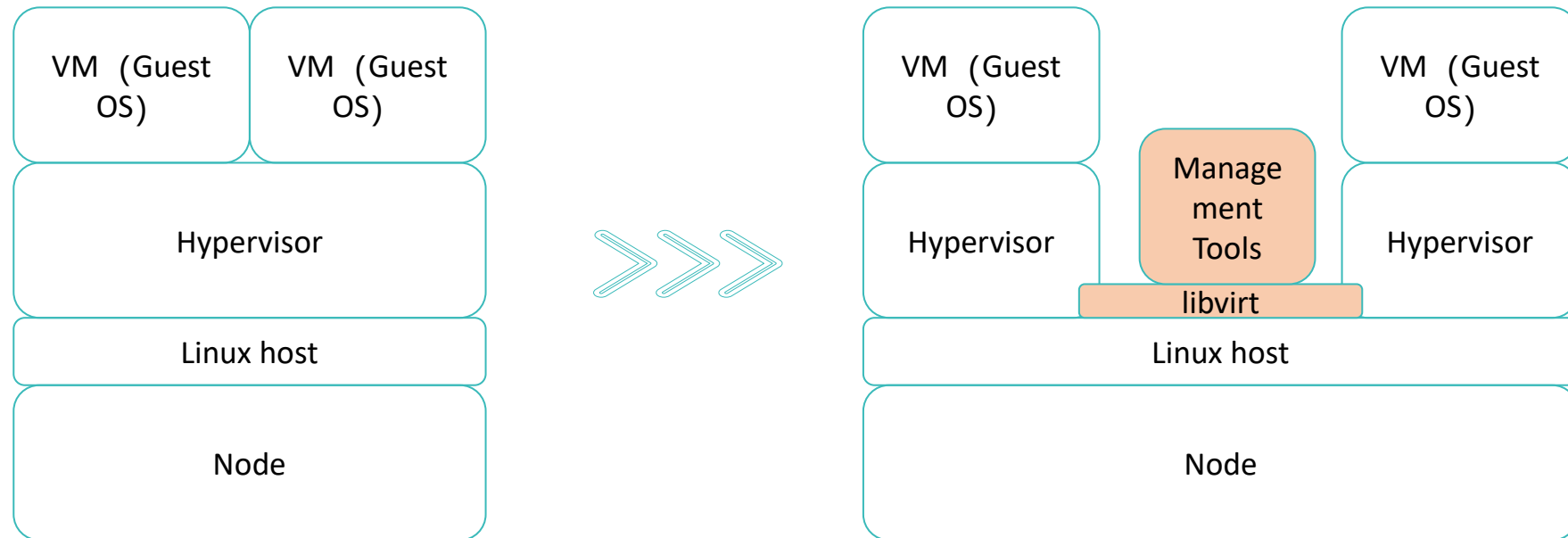
KVM是什么？



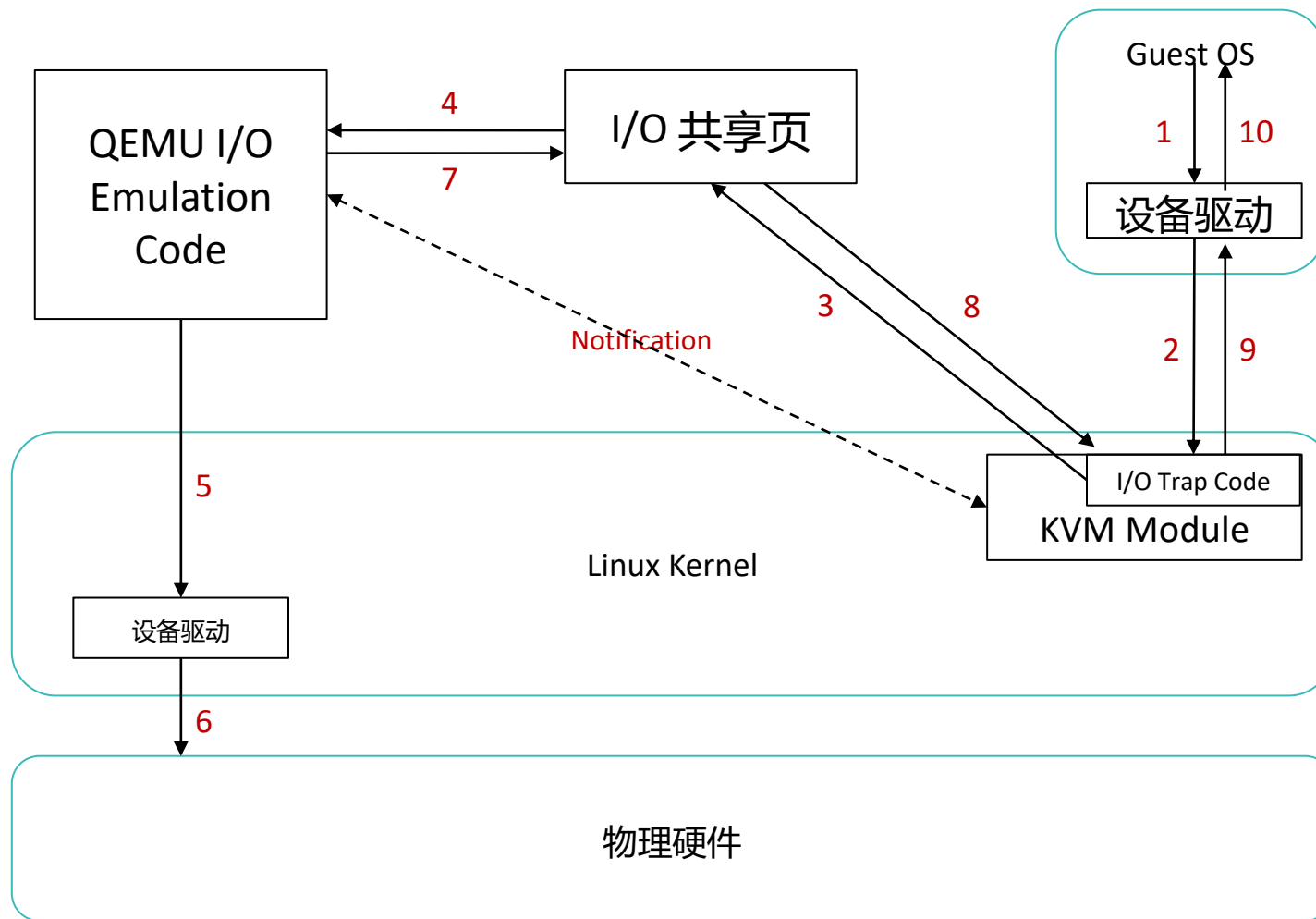
KVM体系架构



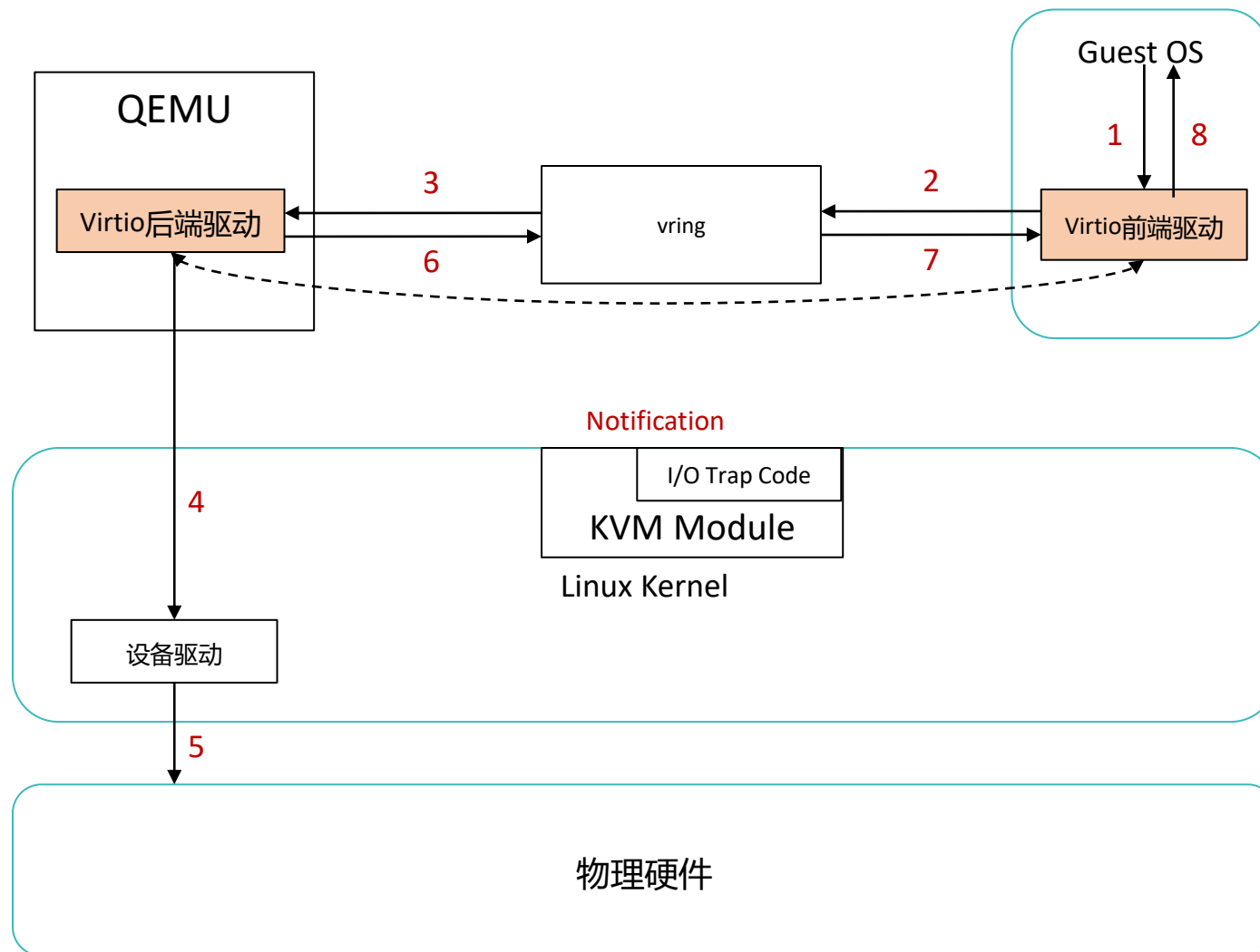
Libvirt



KVM I/O操作流程 - 默认



KVM I/O操作流程 - Virtio



目录

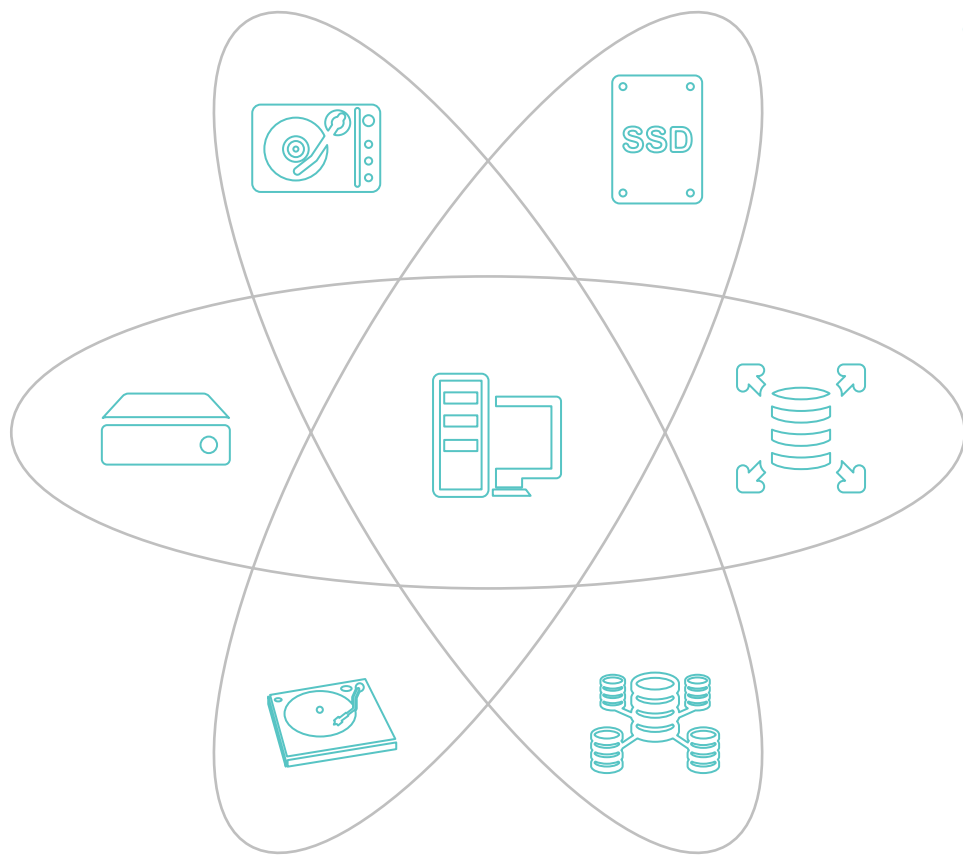
1. 虚拟化技术

- 计算虚拟化
- 网络虚拟化
- 存储虚拟化
- 虚拟化特性

2. OpenStack

3. 容器技术

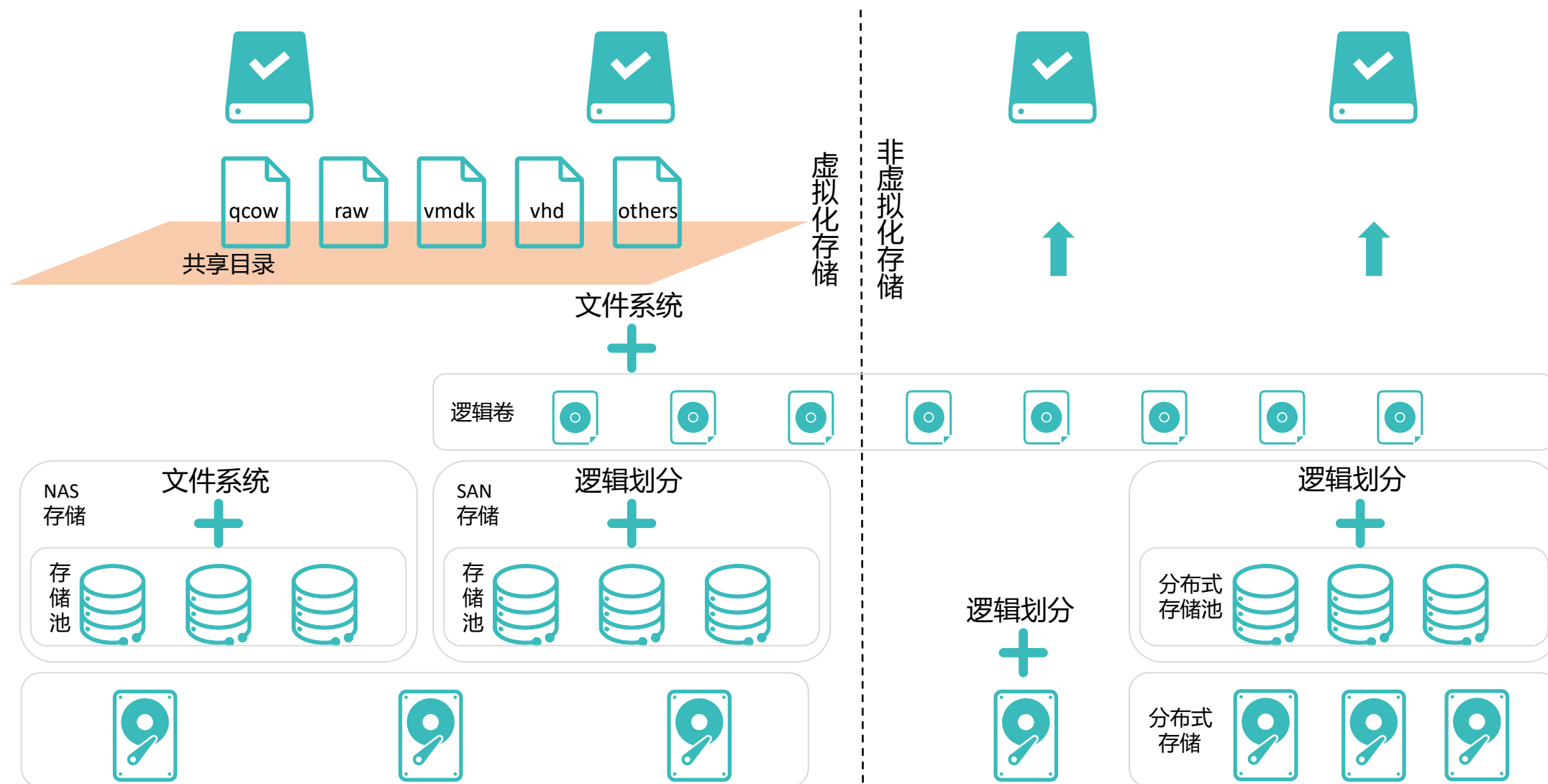
虚拟机使用的存储资源从哪来？



上层的虚拟机要
怎么使用底层这
些存储资源呢？



虚拟化中的存储架构



物理磁盘类型 - SATA盘

- SATA的全称是Serial Advanced Technology Attachment, SATA (Serial ATA) 口的硬盘又叫串口硬盘。SATA采用串行连接方式, 串行ATA总线使用嵌入式时钟信号, 具备了更强的纠错能力, 与以往相比其最大的区别在于能对传输指令 (不仅仅是数据) 进行检查, 如果发现错误会自动矫正, 这在很大程度上提高了数据传输的可靠性。串行接口还具有结构简单、支持热插拔的优点。



物理磁盘类型 - SAS盘

- SAS (Serial Attached SCSI) 即串行连接SCSI，是新一代的SCSI技术，与SATA硬盘相同，都是采用串行技术以获得更高的传输速度，并通过缩短连结线改善内部空间等。SAS是并行SCSI接口之后开发出的全新接口。此接口的设计是为了改善存储系统的效能、可用性和扩充性，并且提供与SATA硬盘的兼容性。

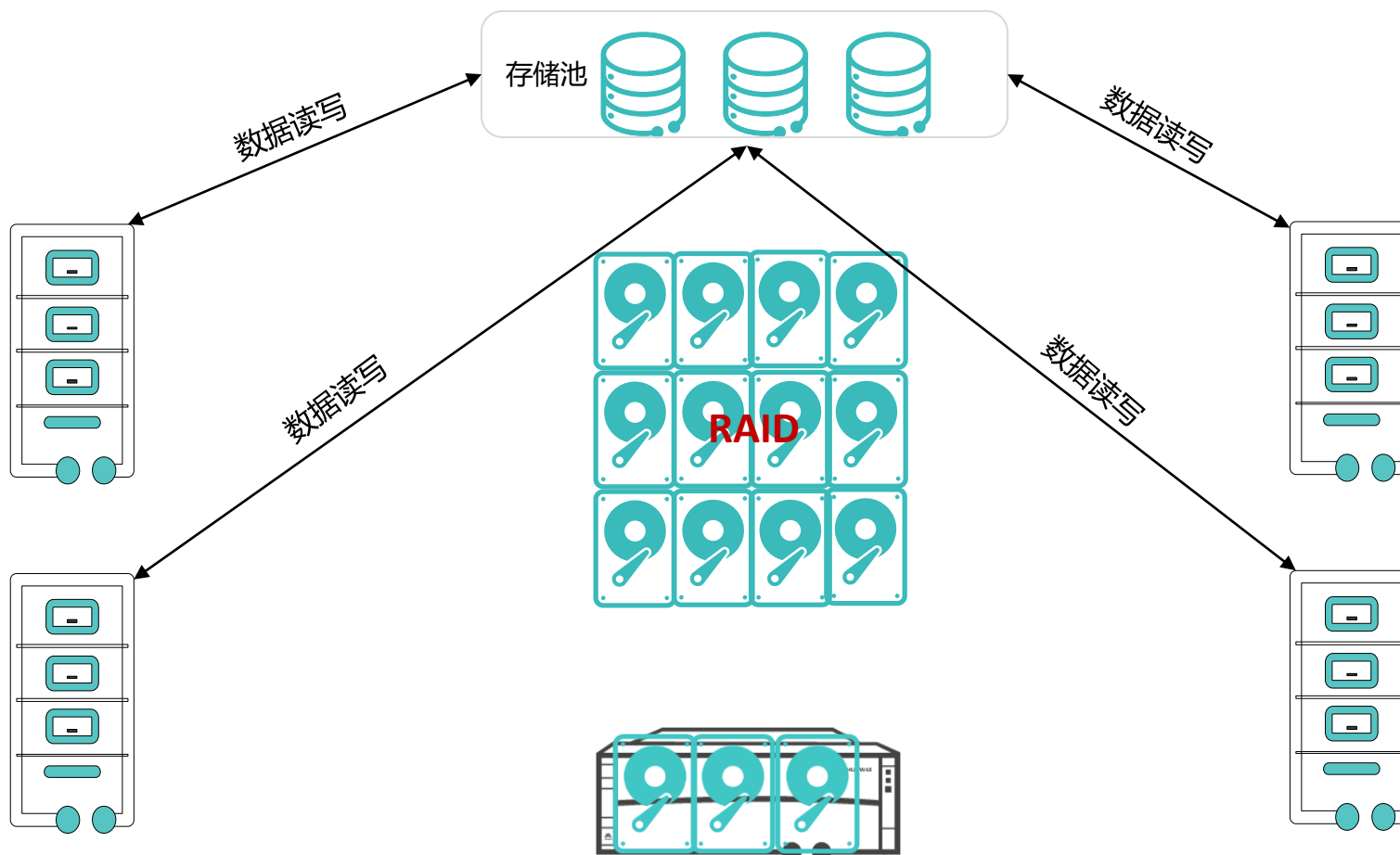


物理磁盘类型 - SSD盘

- SSD (Solid State Disk) , 固态硬盘, 是用固态电子存储芯片阵列而制成的硬盘, 由控制单元和存储单元 (FLASH芯片、DRAM芯片) 组成。SSD在接口的规范和定义、功能及使用方法上与普通硬盘的完全相同, 在产品外形和尺寸上也完全与普通硬盘一致。SSD虽然具有传统机械硬盘所不具备的快速读写、质量轻、能耗低以及体积小等特点, 但其使用寿命有限且价格较高。

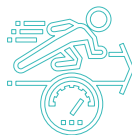
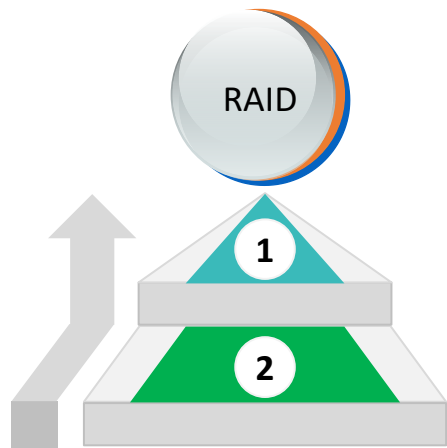


集中式存储



RAID技术

- RAID(独立硬盘冗余阵列)是一种将多个同类型硬盘通过一定的数据组织方式组成容量更大、安全性更高、性能更好的存储的技术。



高效

多块磁盘并行读写

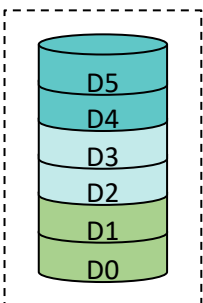


安全

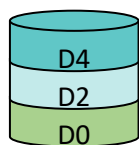
奇偶校验和热备技术

常用RAID类型

逻辑磁盘



D0, D1, D2, D3, D4, D5

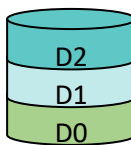
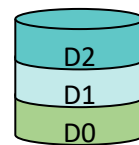


Disk1

Disk2

RAID 0

D0, D1, D2

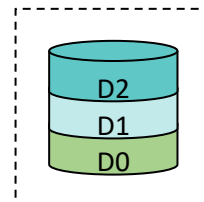


Disk1

Disk2

RAID 1

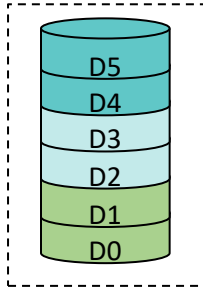
逻辑磁盘



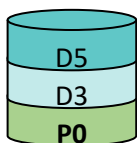
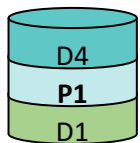
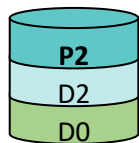
RAID 5

RAID 6

逻辑磁盘



D0, D1, D2, D3, D4, D5

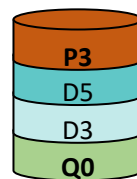
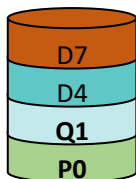
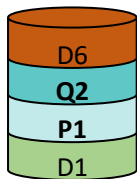
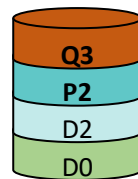


Disk1

Disk2

Disk3

D0, D1, D2, D3, D4, D5, D6, D7



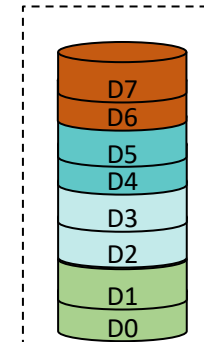
Disk1

Disk2

Disk3

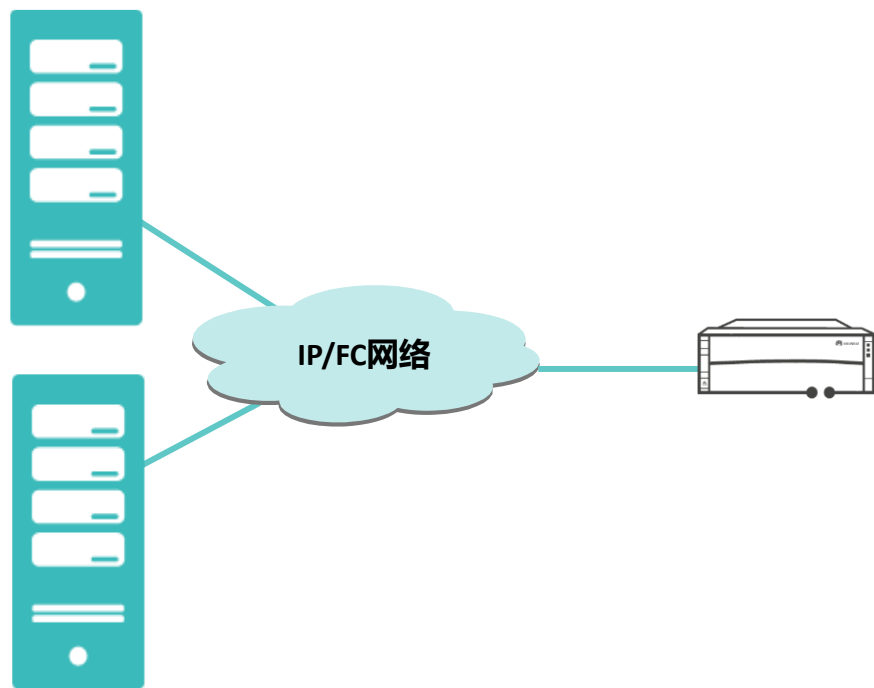
Disk4

逻辑磁盘

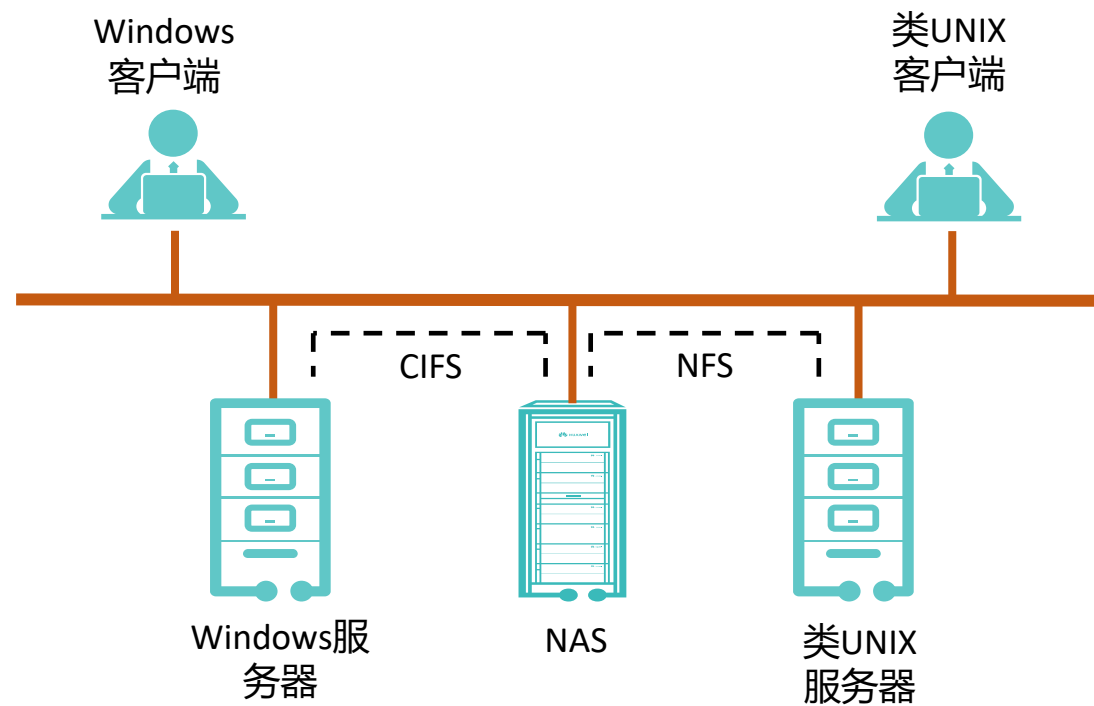


集中式存储类型

SAN (Storage Area Network)

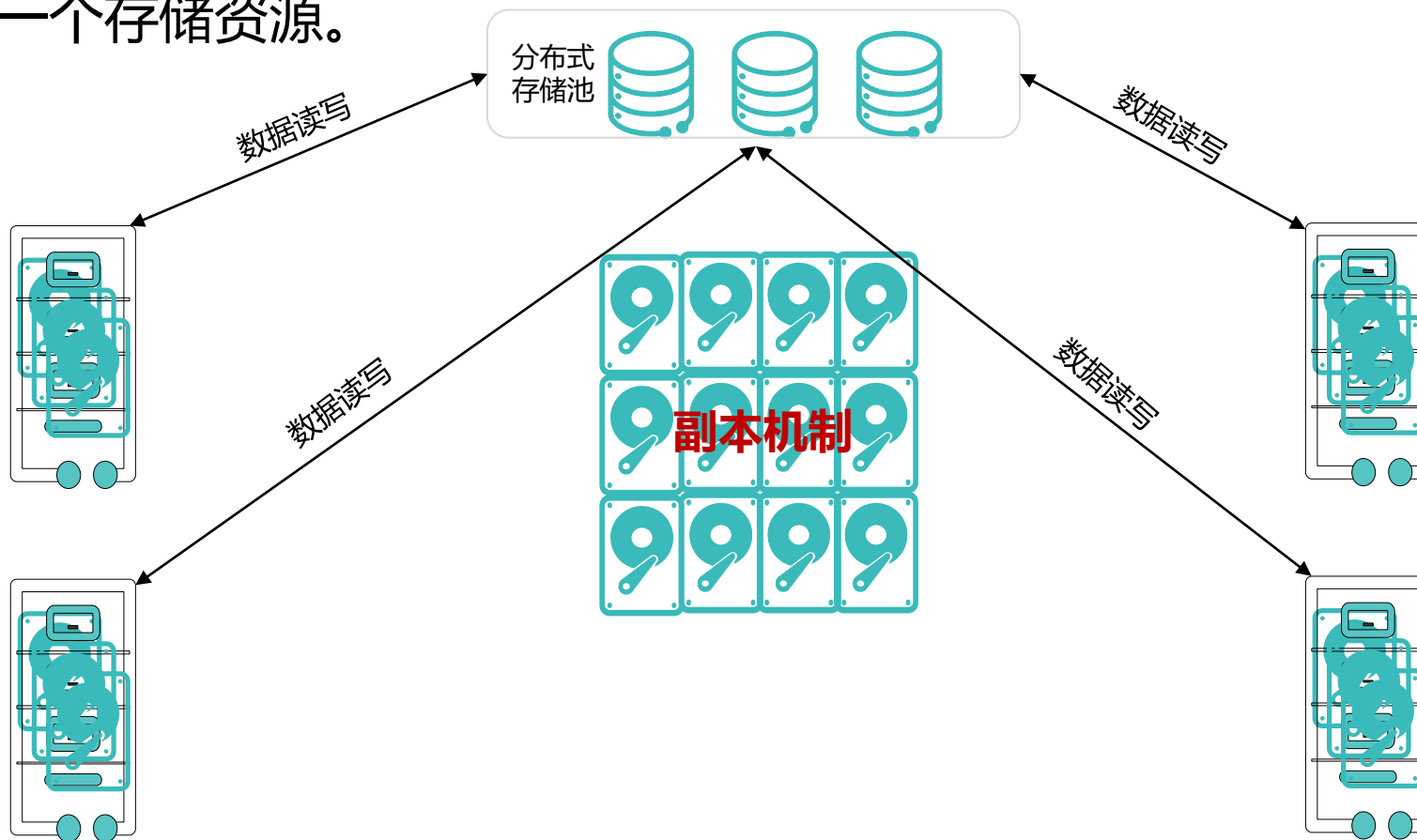


NAS



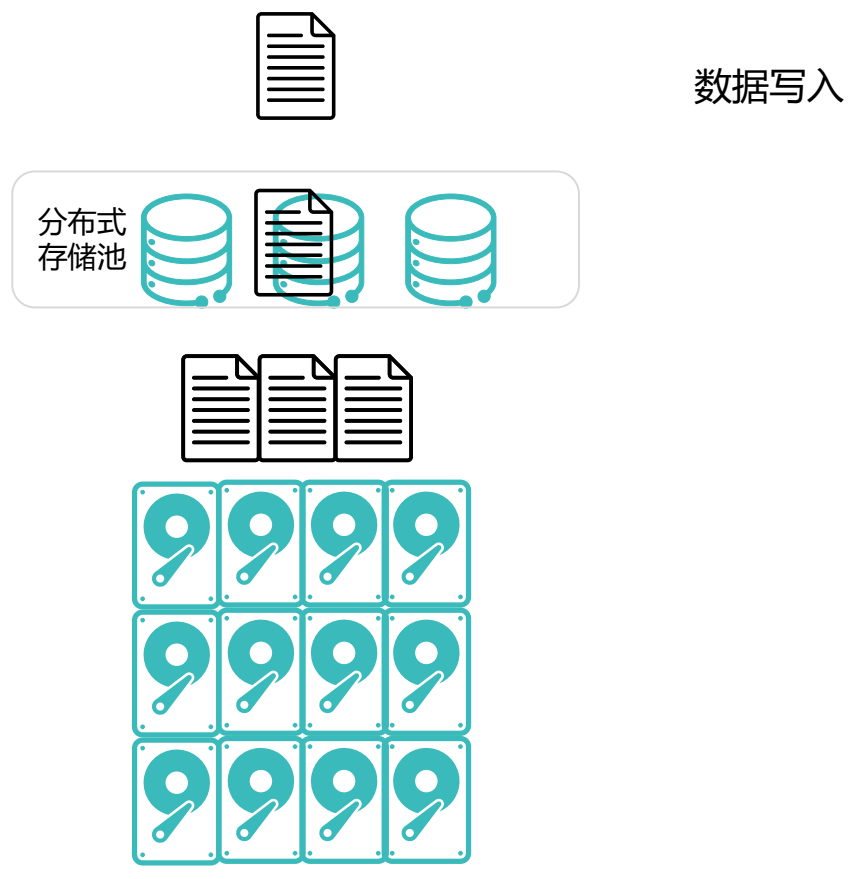
分布式存储

- 不同于集中式存储有独立存储设备，分布式存储通常是通过软件将多台服务器的硬盘整合成一个存储资源。

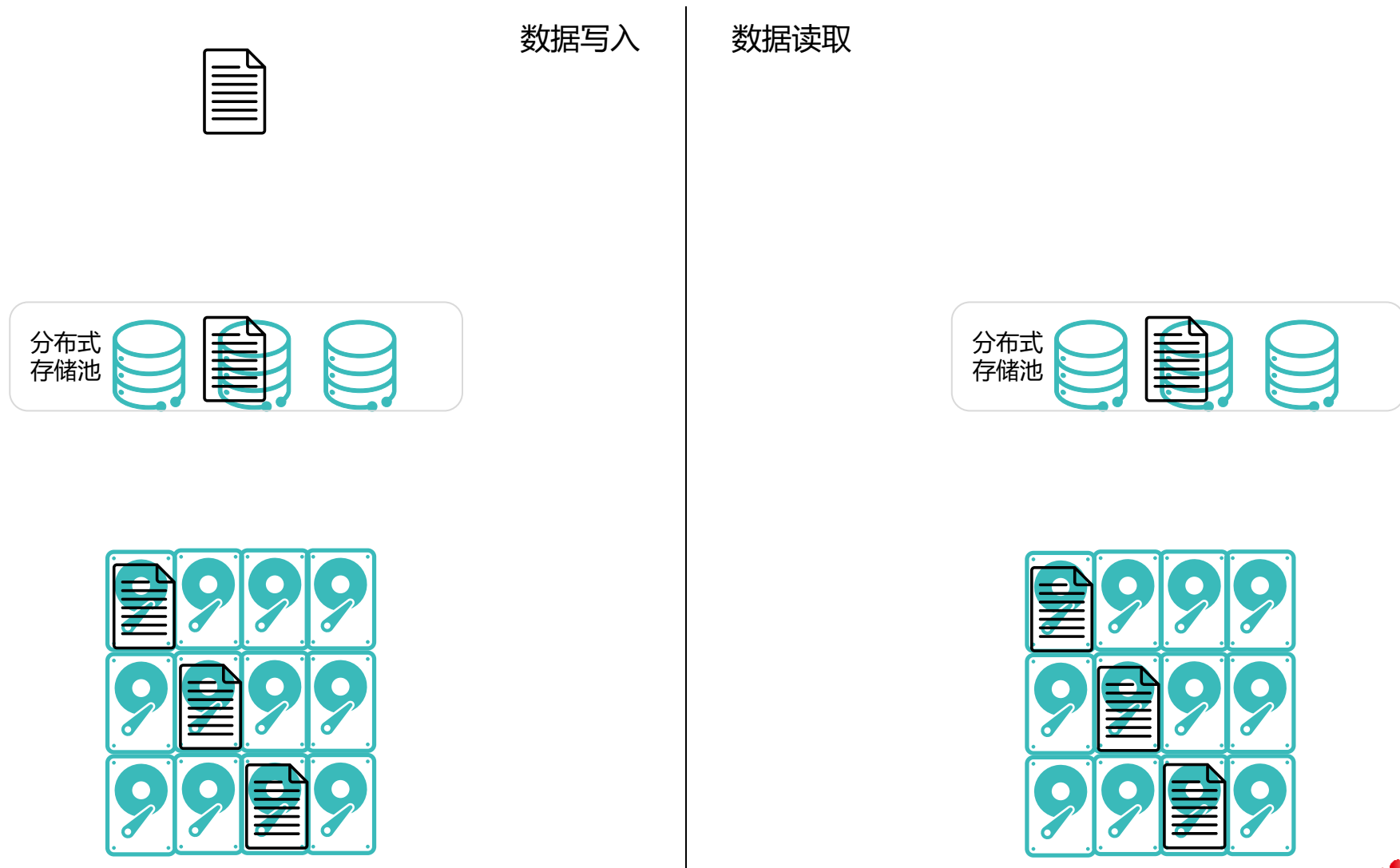


副本机制

- 集中式存储通常使用RAID技术保证数据安全，而分布式存储通常使用副本方式保证数据安全，也有EC算法等技术。



副本机制

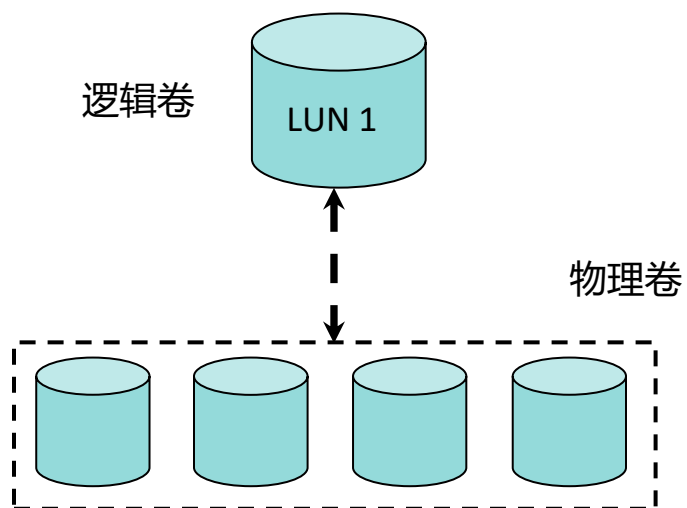


常见分布式存储产品

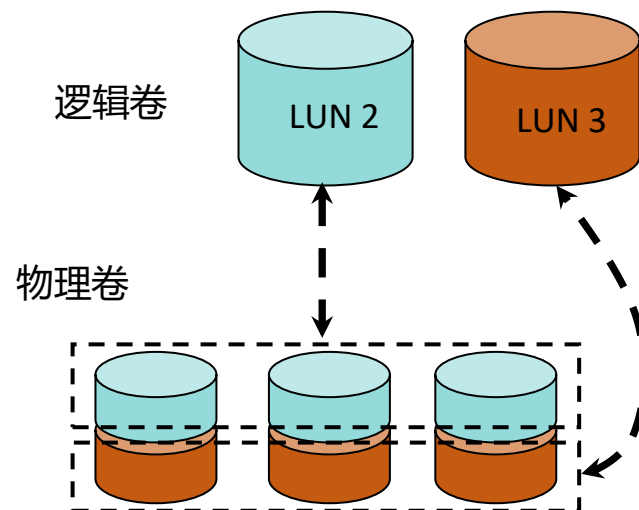


RAID与LUN的关系

- RAID由几个硬盘组成，从整体上看相当于由多个硬盘组成的一个大的物理卷。
- 在物理卷的基础上可以按照指定容量创建一个或多个逻辑单元，这些逻辑单元称作LUN（Logical Unit Number），可以作为映射给主机的基本块设备。

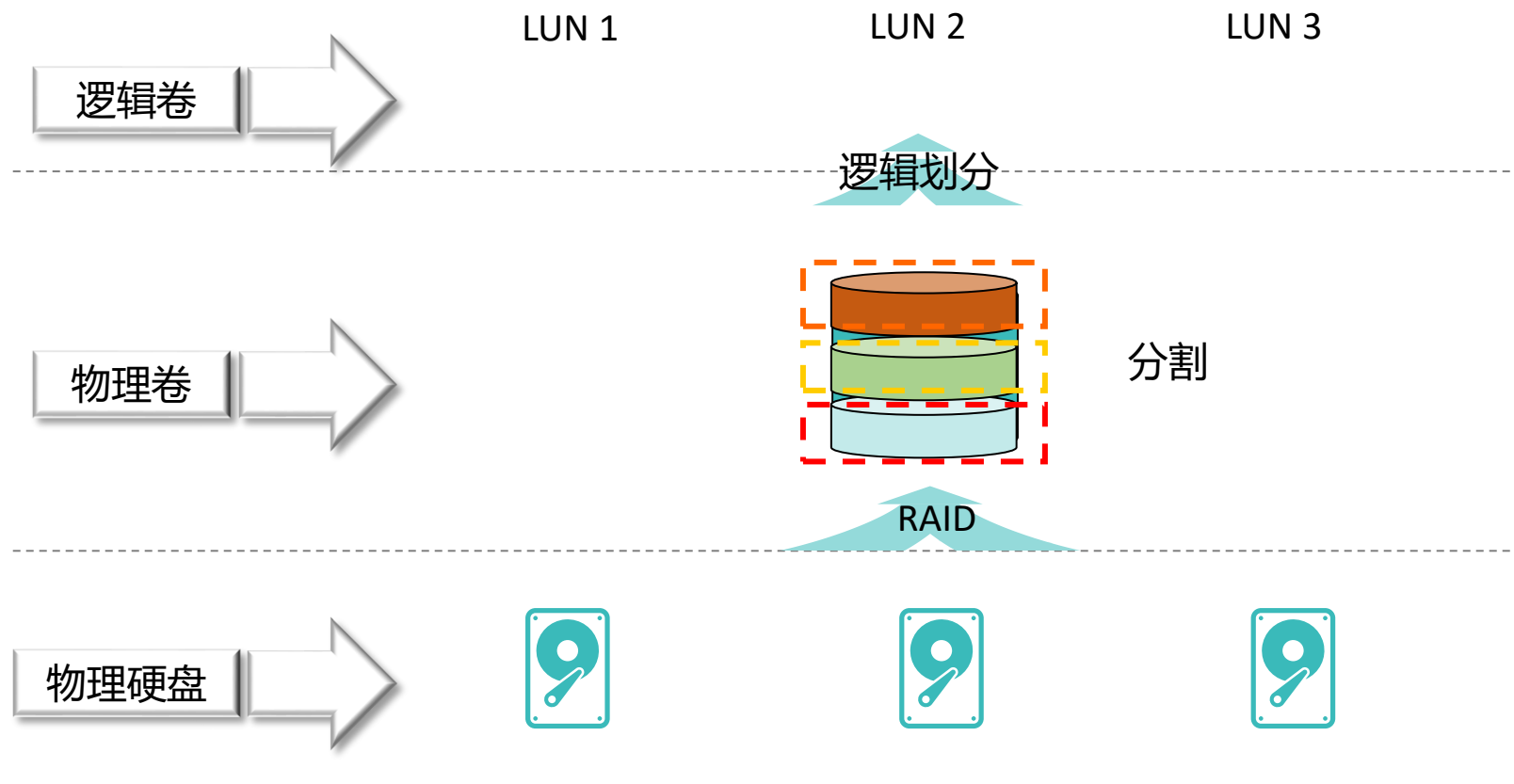


单个物理卷创建一个逻辑卷

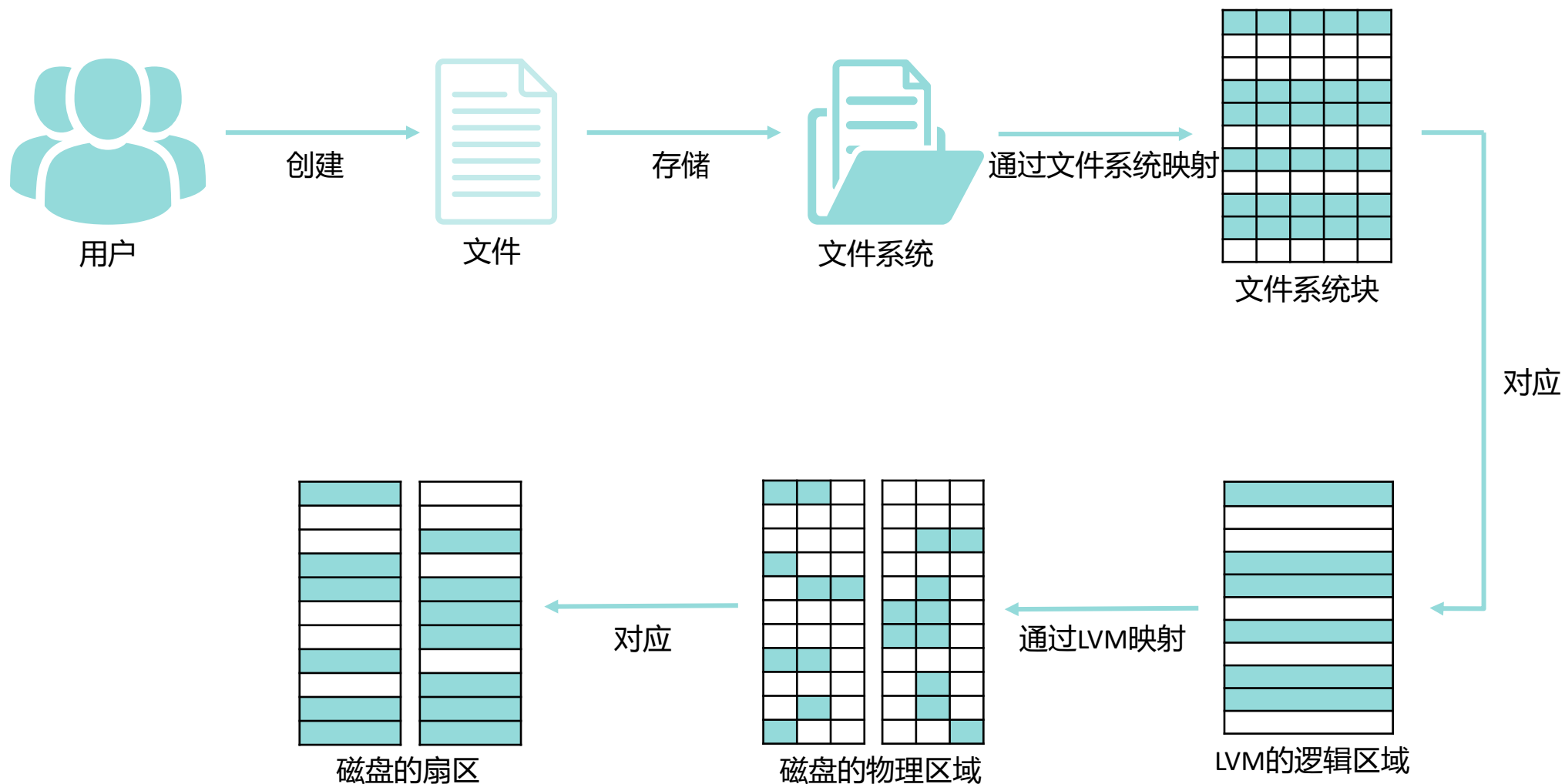


单个物理卷创建多个逻辑卷

创建LUN的过程



文件映射到磁盘的过程



虚拟机磁盘

卷	布局	类型	文件系统	状态	容量	可用空间	% 可用
(C:)	简单	基本	NTFS	状态良好 (启动, 页面文件, 故障转储, 主分区)	100.00 GB	20.76 GB	21 %
(磁盘 0 磁盘分区 1)	简单	基本		状态良好 (恢复分区)	450 MB	450 MB	100 %
(磁盘 0 磁盘分区 2)	简单	基本		状态良好 (EFI 系统分区)	100 MB	100 MB	100 %
DataDisk (D:)	简单	基本	NTFS	状态良好 (主分区)	123.02 GB	24.46 GB	20 %

磁盘 0
基本
223.56 GB
联机

450 MB 状态良好 (恢复分区)	100 MB 状态良好 (EFI 系	(C:) 100.00 GB NTFS 状态良好 (启动, 页面文件, 故障转储, 主分区)	DataDisk (D:) 123.02 GB NTFS 状态良好 (主分区)
-----------------------	-----------------------	--	---

用户视角

管理员视角

```
</disk>
<disk type='file' device='disk'>
  <driver name='qemu' type='raw' cache='none' io='native' iothread='1' />
  <source file='/POME/datastore_3/vol/vol_f6db8fbd-c0c8-4cc4-9252-a862767219d0/vol_f6db8fbd-c0c8-4cc4-9252-a862767219d0.img' />
  <target dev='vda' bus='virtio' />
  <boot order='1' />
  <address type='pci' domain='0x0000' bus='0x02' slot='0x01' function='0x0' />
```

常见虚拟机磁盘格式

虚拟机磁盘文件格式	支持的厂商
RAW	各厂商通用
VMDK	VMware
VHD	微软Hyper-V、华为FusionCompute
QCOW	QEMU或KVM虚拟化平台专用的格式
QED	
VDI	Oracle

目录

1. 虚拟化技术

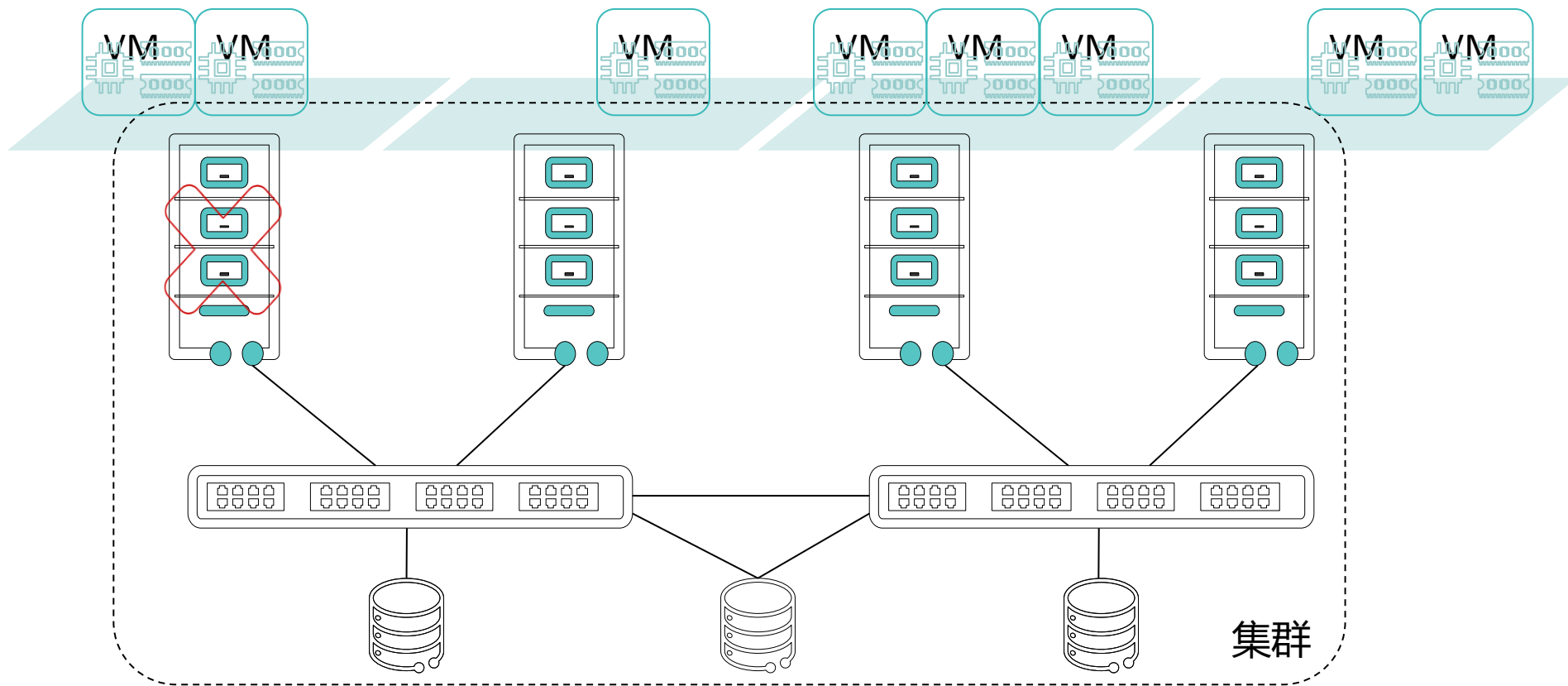
- 计算虚拟化
- 网络虚拟化
- 存储虚拟化
- 虚拟化特性

2. OpenStack

3. 容器技术

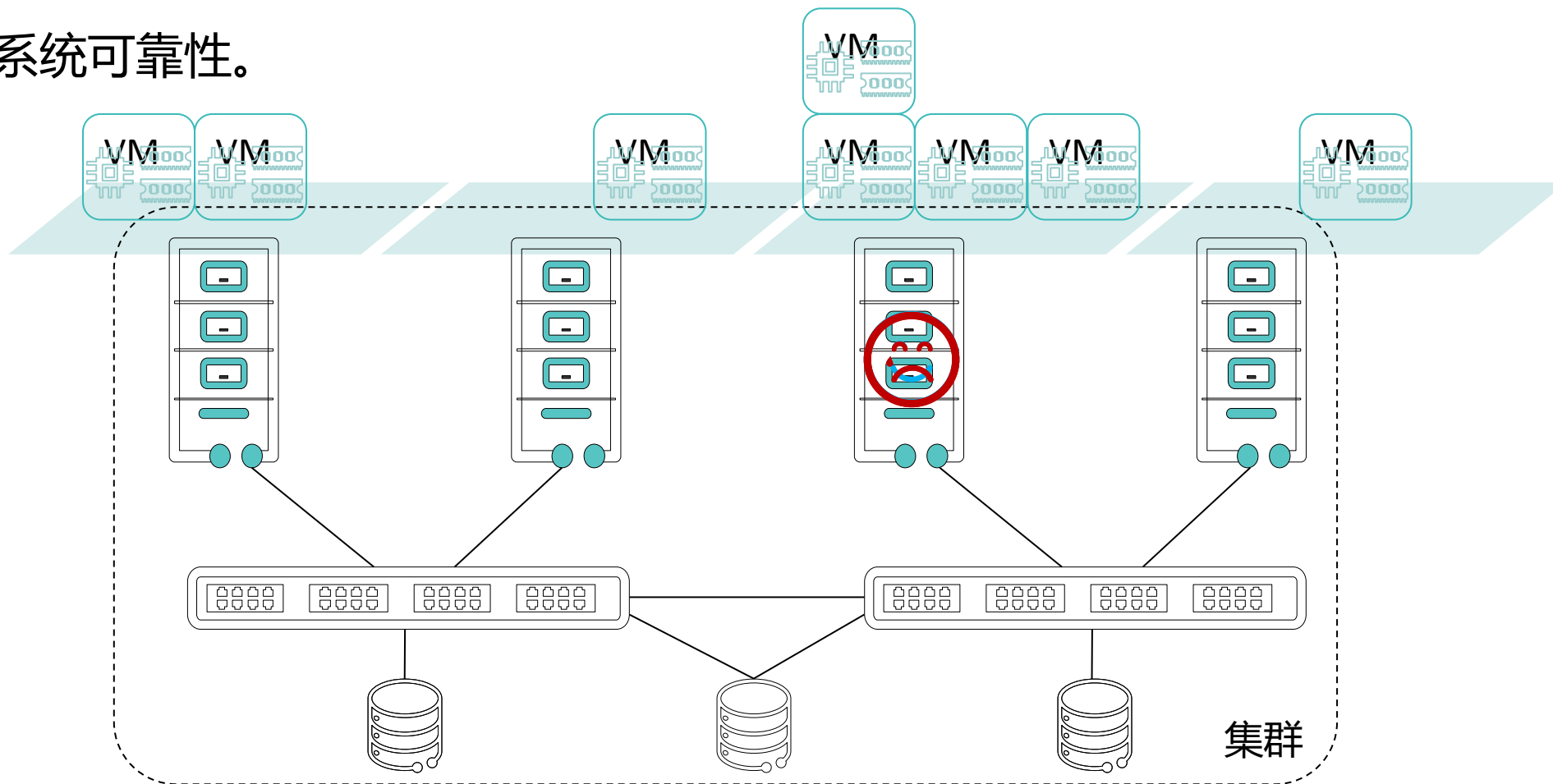
虚拟化集群特性 - 虚拟机HA

- 虚拟机HA功能可以保障在主机发生故障时的业务连续性。

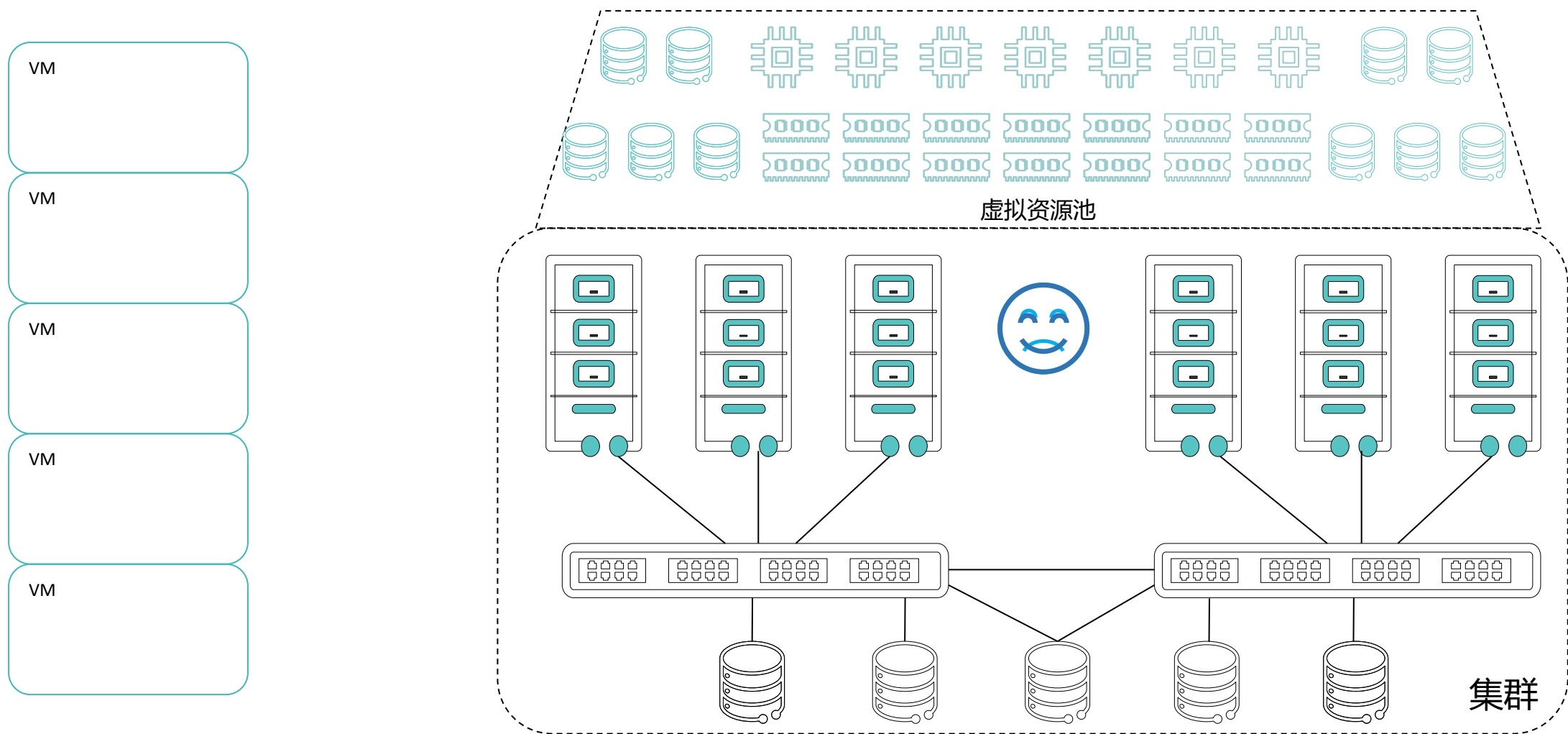


虚拟化集群特性 - 负载均衡

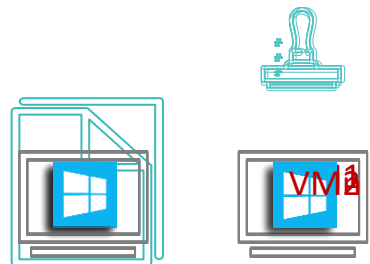
- 利用集群的负载均衡特性动态调整虚拟机所属主机，保障主机资源利用率尽量一致，提高系统可靠性。



虚拟化集群特性 - 易扩容



虚拟机特性 - 按模板部署虚拟机与批量复制



VM1



VM2



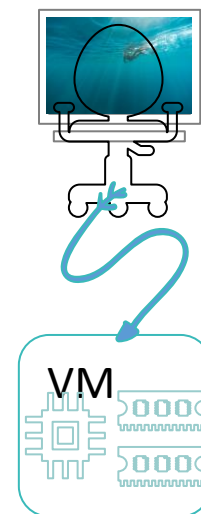
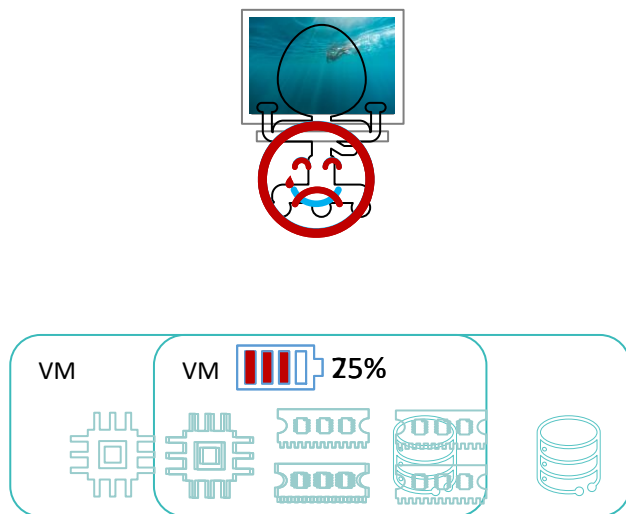
VM3



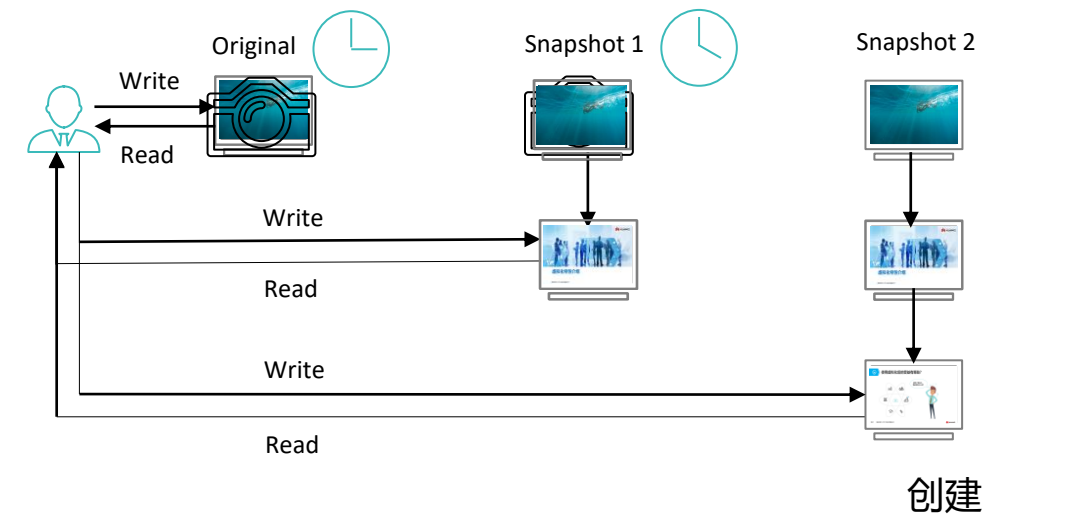
VM4



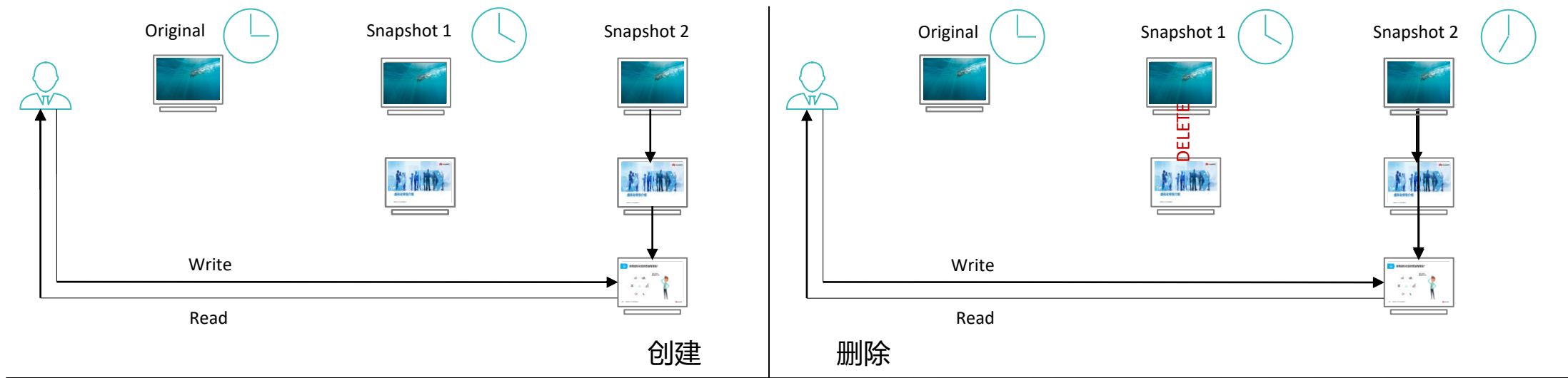
虚拟机特性 - 资源热添加与虚拟控制台



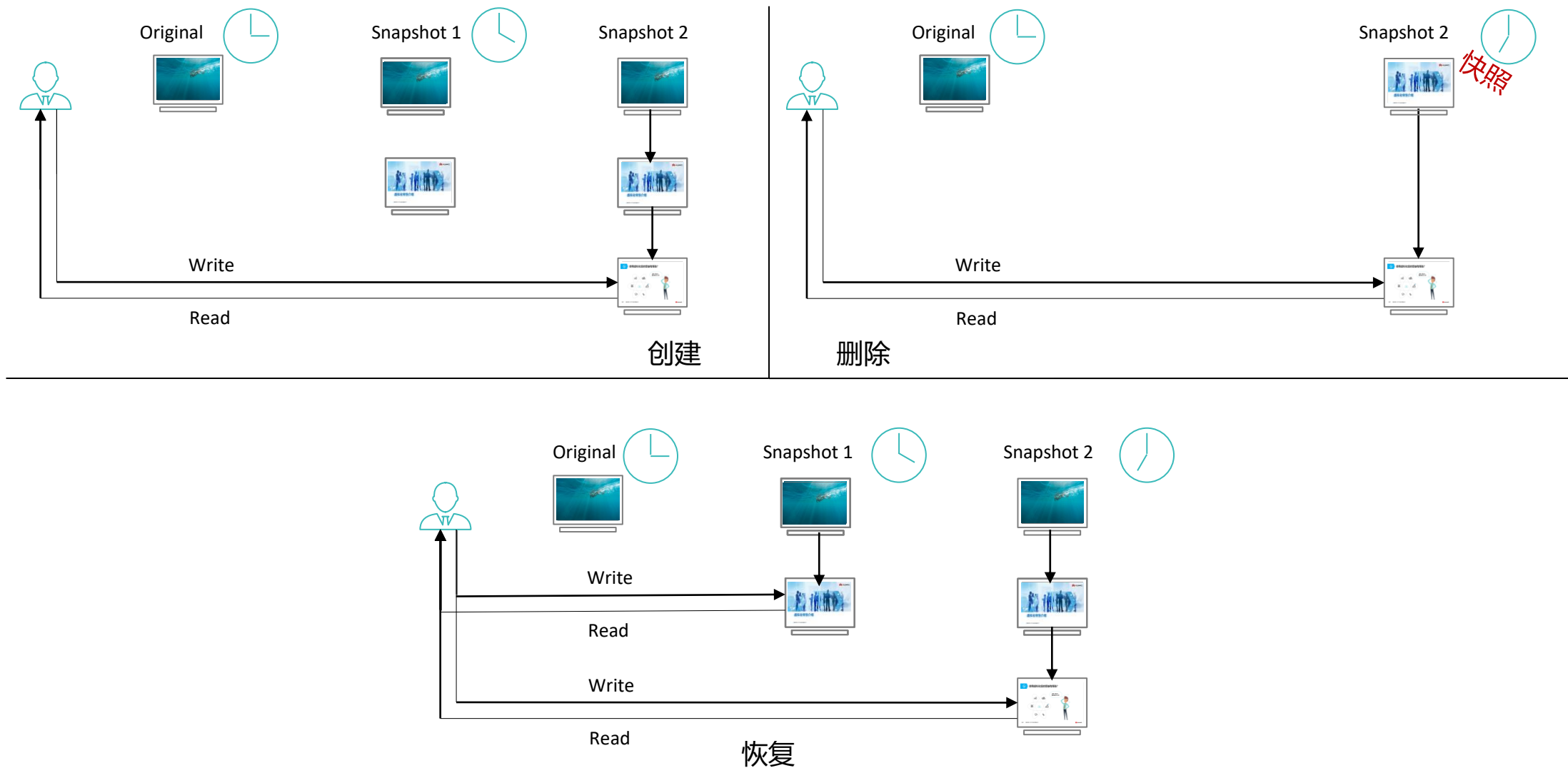
虚拟机特性 - 快照 (1)



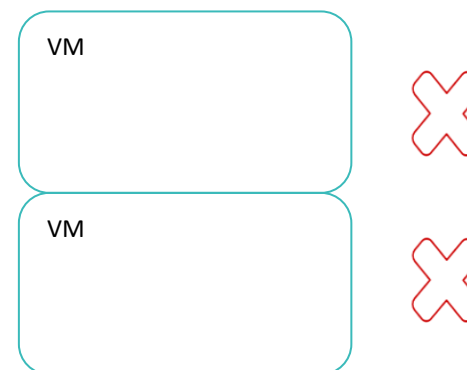
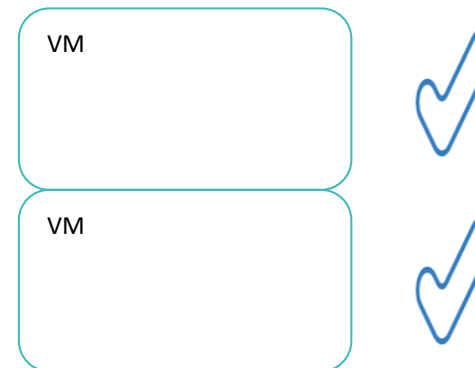
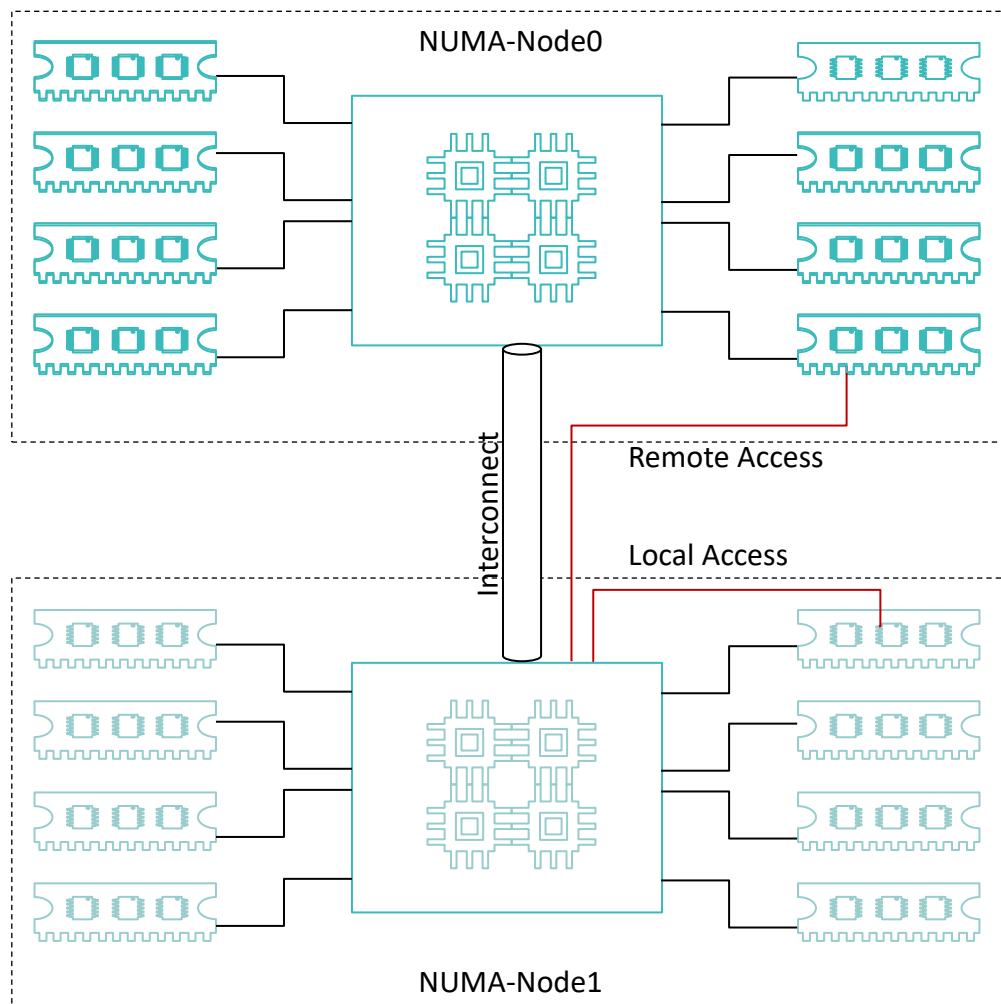
虚拟机特性 - 快照 (2)



虚拟机特性 - 快照 (3)



NUMA



更多信息

- 本节简要介绍虚拟化技术基本内容，详细内容请参考HCIA-Cloud Computing认证教材及实验手册，资料获取链接：
https://e.huawei.com/cn/talent/#/cert/product-details?certifiedProductId=837&authenticationLevel=CTYPE_CARE_HCIA&technicalField=PSC&version=5.5

目录

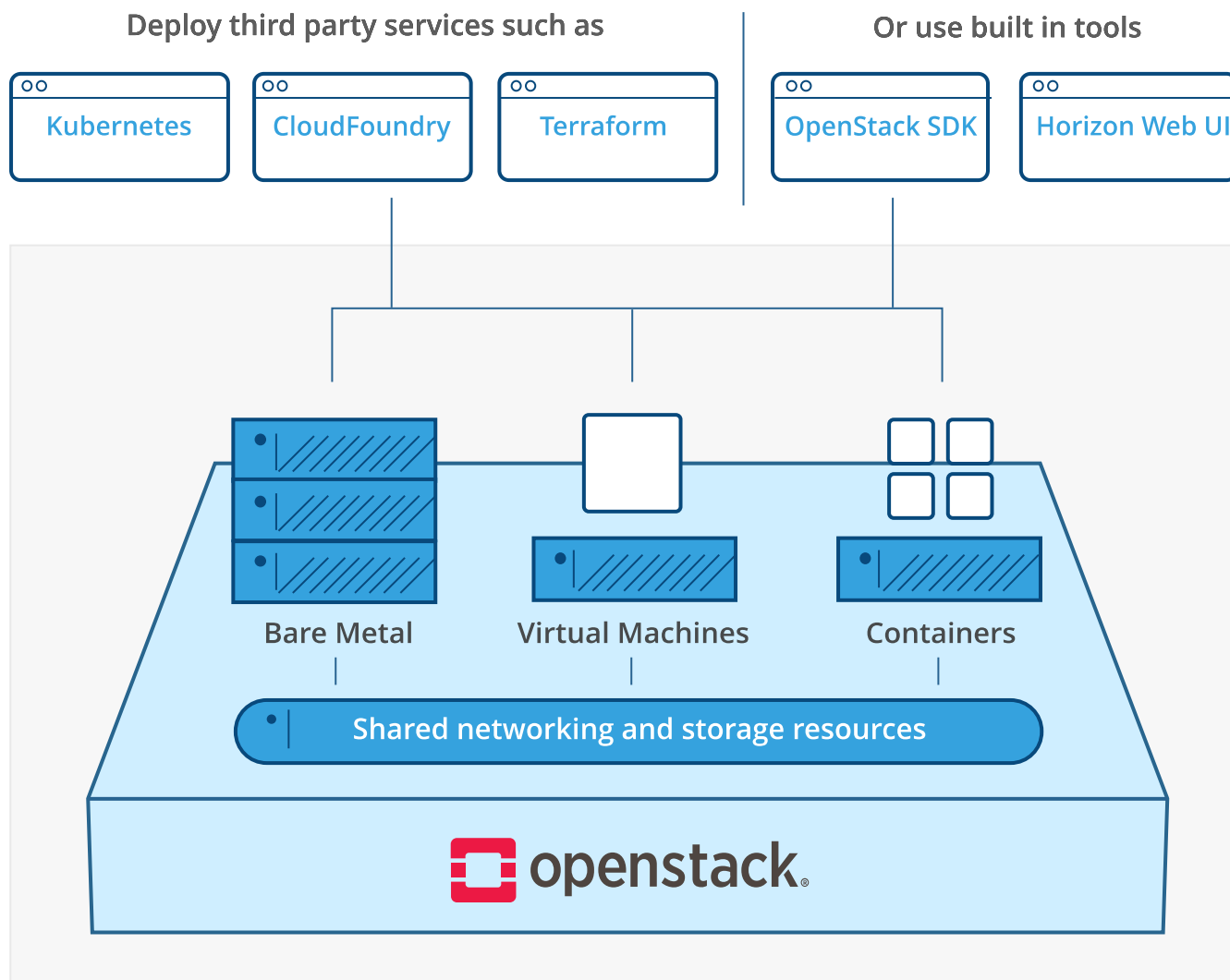
1. 虚拟化技术

2. **OpenStack**

- OpenStack简介
- OpenStack架构
- OpenStack核心服务简介
- OpenStack服务间交互示例

3. 容器技术

OpenStack是什么？



OpenStack是开源云计算平台，可控制整个数据中心的大型计算，存储和网络资源池。

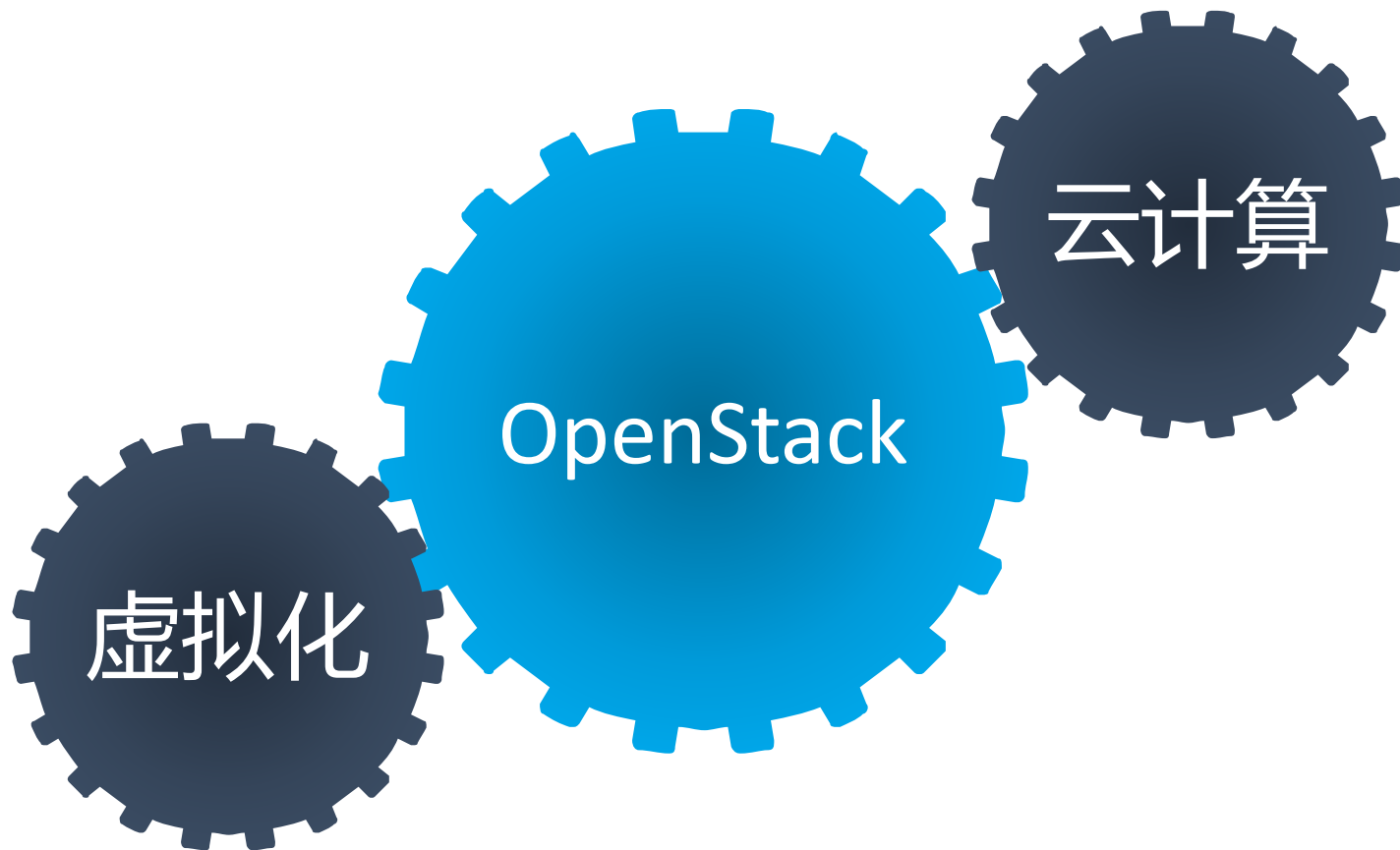
用户能够通过Web界面、命令行或API接口配置资源。

source: www.openstack.org



OpenStack和虚拟化、云计算什么关系？

- 请花五分钟时间思考或讨论：OpenStack和虚拟化、云计算什么关系？



OpenStack不是虚拟化

- OpenStack的架构定位与技术范畴：
 - OpenStack只是系统的控制面。
 - OpenStack不包括系统的数据面组件，如Hypervisor、存储和网络设备等。
- OpenStack和虚拟化有着关键的区别：

OpenStack

- 自身不提供虚拟化技术
- 调用多种技术实现多资源池管理
- 对外提供统一管理接口

VS

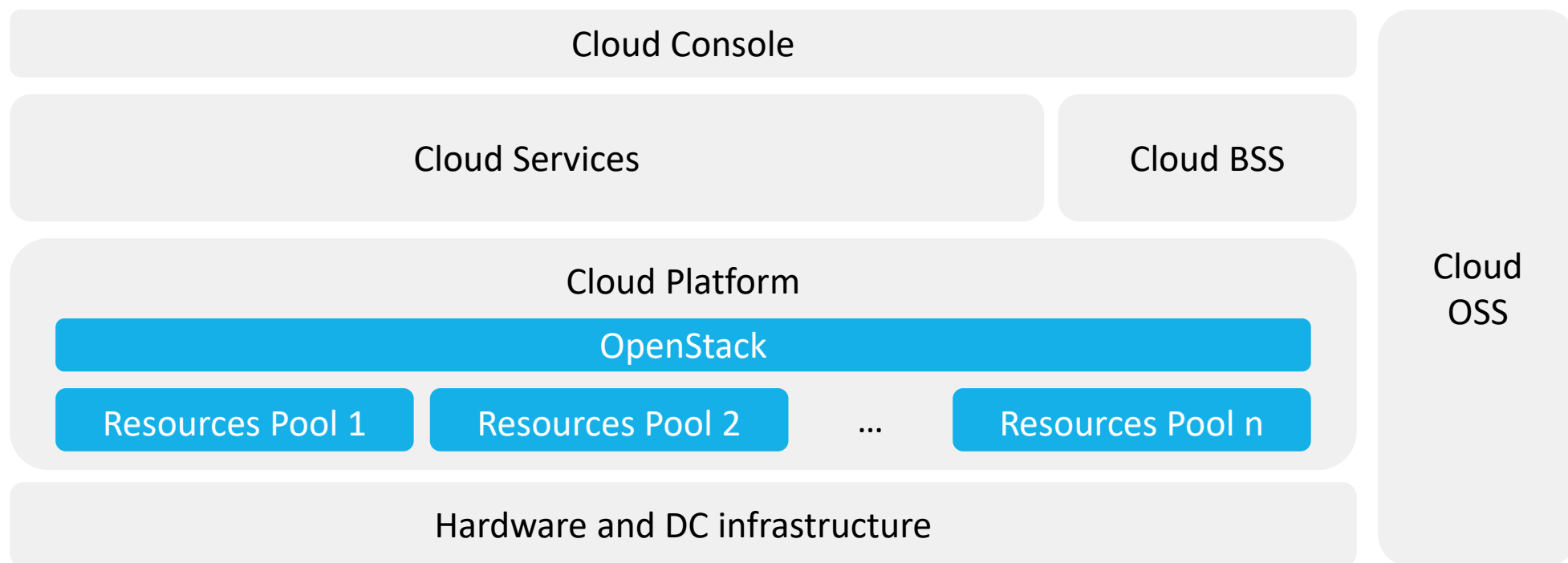
虚拟化

- 环境隔离，资源复用
- 降低隔离损耗，提升运行效率
- 提供高级虚拟化特性

虚拟化是OpenStack底层的技术实现手段之一，但并非核心关注点。

OpenStack不是云计算

- OpenStack只是构建云计算的关键组件：
 - 内核、骨干、框架、总线
- 为了构建云计算，我们还需要很多东西：



OpenStack的设计思想

开放

- 开源，并尽可能重用已有开源项目
- 不要“重复发明轮子”

灵活

- 不使用任何不可替代的私有/商业组件
- 大量使用插件方式进行架构设计与实现

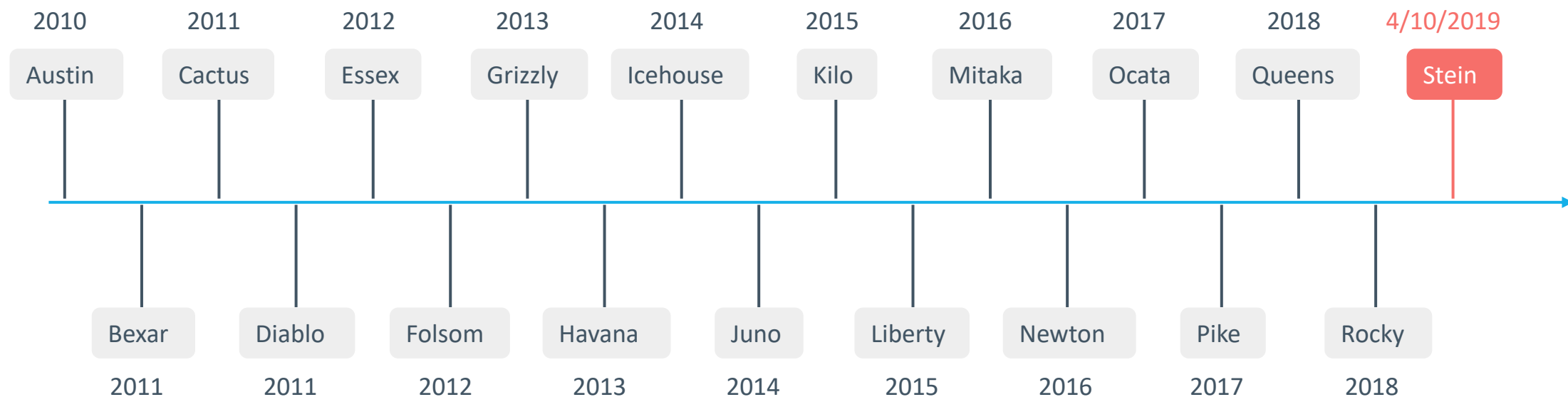
可扩展

- 由多个相互独立的项目组成
- 每个项目包含多个独立服务组件
- 无中心架构
- 无状态架构

- Apache 2.0 License
- 约70%的代码（核心逻辑）使用Python开发

OpenStack历史版本

- OpenStack每年两个大版本，一般在4月和10月中旬发布，版本命名从字母A-Z。



目录

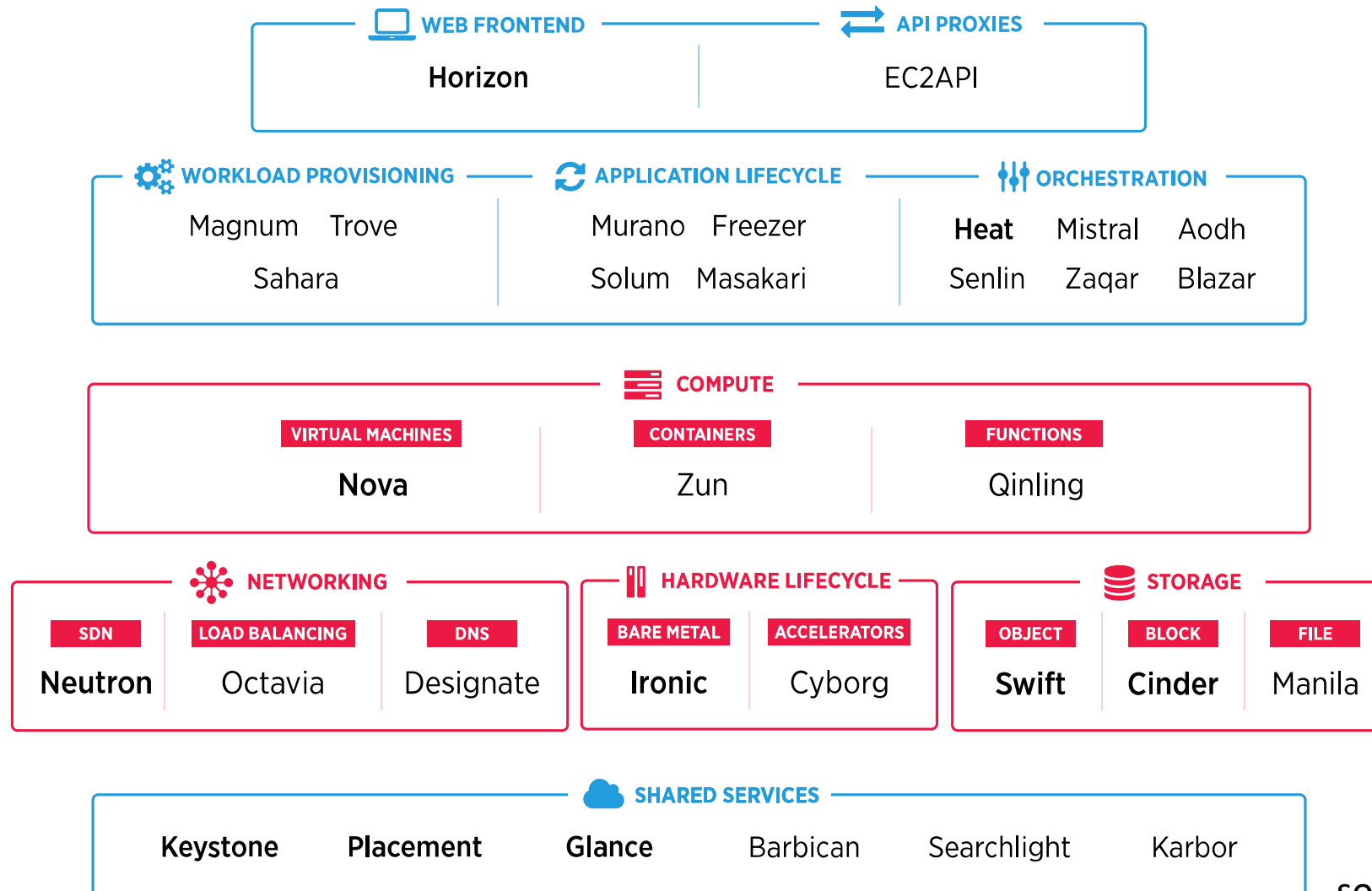
1. 虚拟化技术

2. **OpenStack**

- OpenStack简介
- OpenStack架构
- OpenStack核心服务简介
- OpenStack服务间交互示例

3. 容器技术

OpenStack架构图



目录

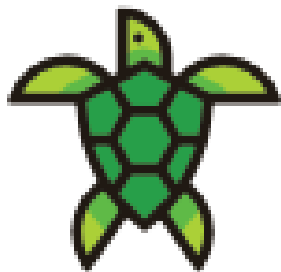
1. 虚拟化技术

2. **OpenStack**

- OpenStack简介
- OpenStack架构
- OpenStack核心服务简介
- OpenStack服务间交互示例

3. 容器技术

认证服务Keystone



KEYSTONE

认证服务

首次出现在OpenStack的“Essex”版本中。

简介

Keystone提供身份验证，服务发现和分布式多租户授权。

Keystone支持LDAP，OAuth，OpenID Connect，SAML和SQL。

OpenStack其他服务依赖性

为其他OpenStack服务提供认证支持。

镜像服务Glance



GLANCE

镜像服务

首次出现在OpenStack的 “Bexar” 版本中。

简介

Glance提供发现、注册和检索虚拟机镜像功能。

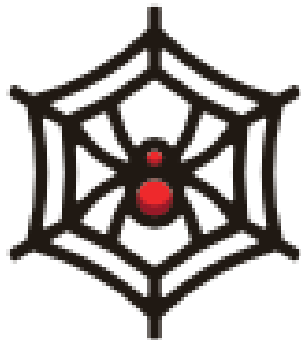
Glance提供的虚拟机实例镜像可以存放在不同地方，例如本地文件系统、Swift对象存储、Cinder块存储等。

OpenStack其他服务依赖性



Keystone

网络服务Neutron



NEUTRON

网络服务

首次出现在OpenStack的“Folsom”版本中。

简介

Neutron负责管理虚拟网络组件，专注于为OpenStack提供网络即服务。

OpenStack其他服务依赖性



Keystone

块存储服务Cinder



CINDER

块存储服务

首次出现在OpenStack的“Folsom”版本中。

简介

Cinder提供块存储服务，为虚拟机实例提供持久化存储。

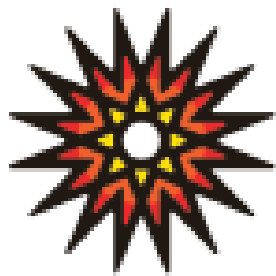
Cinder调用不同存储接口驱动，将存储设备转化成块存储池，用户无需了解存储实际部署的位置或设备类型。

OpenStack其他服务依赖性



Keystone

计算服务Nova



NOVA

计算服务

首次出现在OpenStack的“Austin”版本中。

简介

Nova提供大规模、可扩展、按需自助服务的计算资源。

Nova支持管理裸机，虚拟机和容器。

OpenStack其他服务依赖性



Keystone



Neutron



Glance

对象存储服务Swift



SWIFT

对象存储服务

首次出现在OpenStack的“Austin”版本中。

简介

Swift提供高度可用、分布式、最终一致的对象存储服务。

Swift可以高效、安全且廉价地存储大量数据。

Swift非常适合存储需要弹性扩展的非结构化数据。

OpenStack其他服务依赖性

为其他OpenStack服务提供对象存储服务。

编排服务Heat



HEAT

编排服务

首次出现在OpenStack的“Havana”版本中。

简介

Heat为云应用程序编排OpenStack基础架构资源。

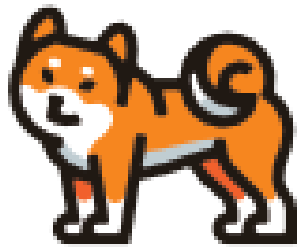
Heat提供OpenStack原生Rest API和CloudFormation兼容的查询API。

OpenStack其他服务依赖性



Keystone

操作界面Horizon



HORIZON

操作界面

首次出现在OpenStack的“Essex”版本中。

简介

Horizon提供基于Web的控制界面，使云管理员和用户能够管理各种OpenStack资源和服务。

OpenStack其他服务依赖性



Keystone

目录

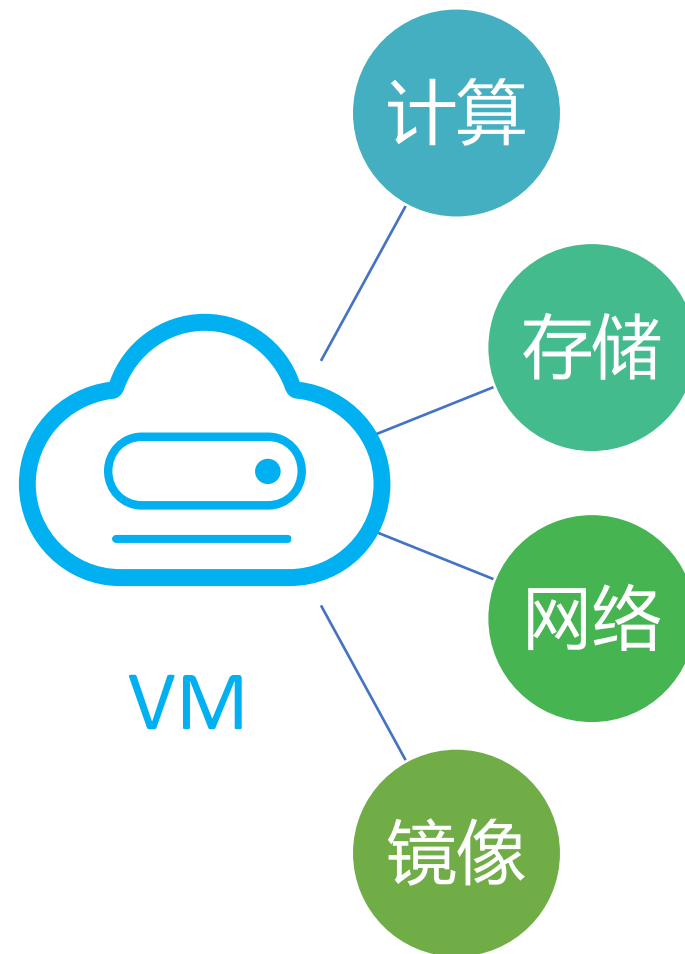
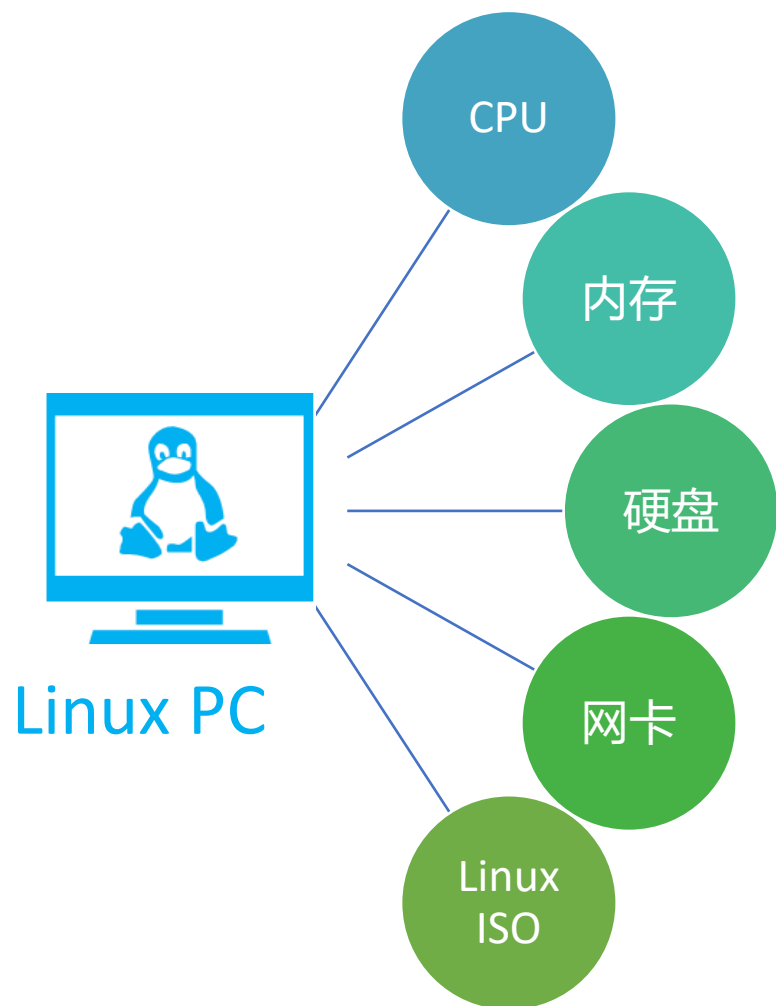
1. 虚拟化技术

2. **OpenStack**

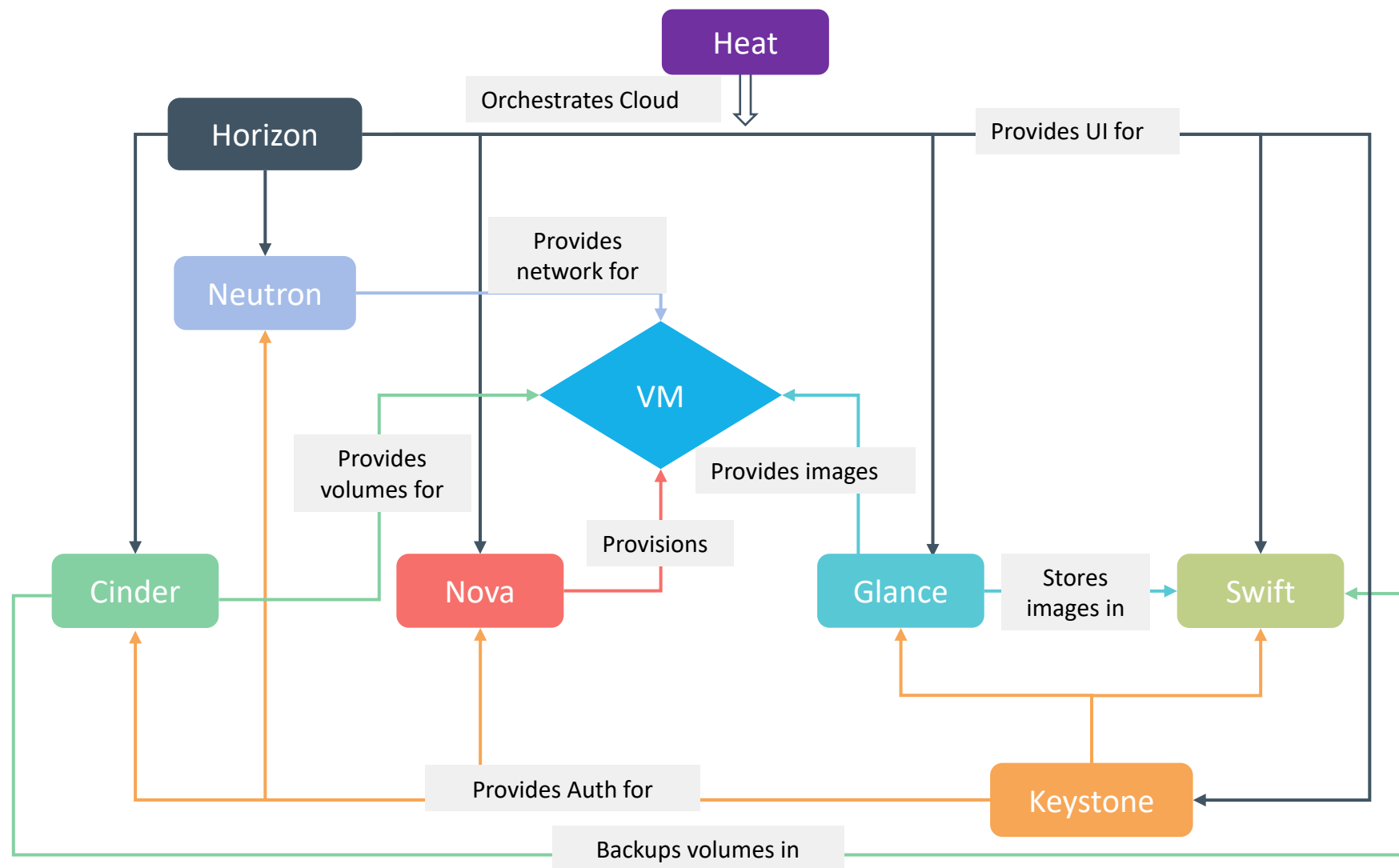
- OpenStack简介
- OpenStack架构
- OpenStack核心服务简介
- OpenStack服务间交互示例

3. 容器技术

提问：创建一个VM需要些什么资源？



OpenStack创建VM，服务间交互示例



更多信息

- 本节简要介绍OpenStack内容，详细内容请参考HCIP-Cloud Computing-OpenStack认证教材及实验手册，资料获取链接：
https://e.huawei.com/cn/talent/#/cert/product-details?certifiedProductId=653&authenticationLevel=CTYPE_CARE_HCIP&technicalField=PSC&version=5.0

目录

1. 虚拟化技术
2. OpenStack
- 3. 容器技术**
 - 容器技术发展
 - 容器技术基础
 - 容器编排技术

应用上云的痛点

- 云计算重构了ICT系统，给社会各行业带来了极大的变革、便利。



- 但云计算同样带来了新的问题：业务怎么上云？
 - 应用在云端重新部署：以脚本或手工方式在云端重新部署。
 - 打包本地已部署应用的系统镜像，通过P2V/V2V等方式上传到云端运行。

Cloud Foundry项目介绍

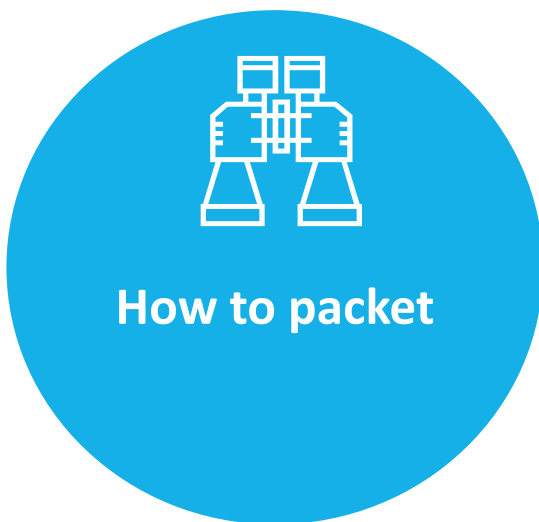
- Cloud Foundry等传统开源PaaS项目的作用
 - 用于解决大规模的应用“上云”的问题。Cloud Foundry项目提供了“**应用托管**”能力，其核心组件是一套应用的打包和分发机制。
- Cloud Foundry项目的实现过程：
 - **自动上传、运行应用包**：运维人员需在本地物理机或虚拟机上部署一个Cloud Foundry项目。开发人员执行简单命令后，即可将本地应用（应用的可执行文件和启动脚本）打包进一个压缩包内，上传到Cloud Foundry云端存储着。Cloud Foundry会通过调度器选择一个可运行该应用的虚拟机并通知其agent下载该应用的压缩包，然后启动运行应用。
 - **为应用提供隔离的运行环境**：由于一个虚拟机中可能需运行来自多个用户的不同应用，Cloud Foundry会调用Namespace和Cgroup为每个应用创建隔离的运行环境（沙盒），以实现应用间互不干涉。在这点上，与Docker Container的实现类似。



SandBox

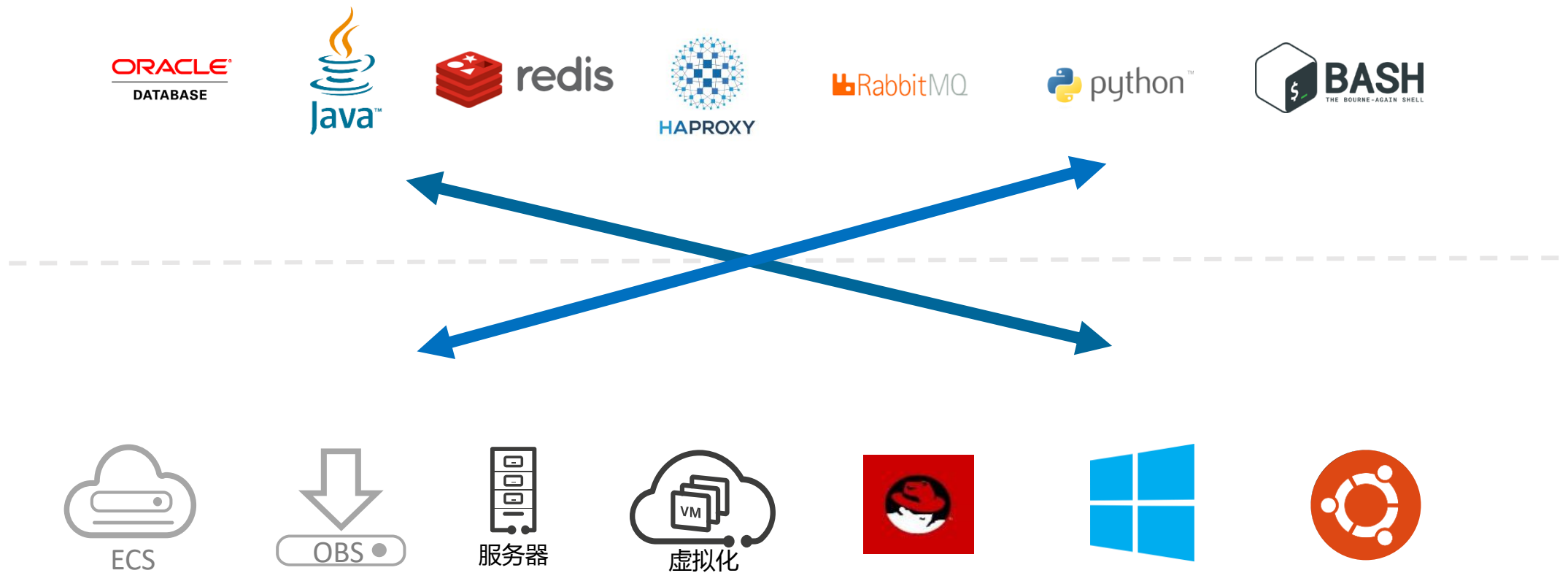
传统PaaS项目的痛点

- 传统开源PaaS项目中的问题：
 - 应用打包困难：因本地环境与云端环境不一致，用户须为每种语言、框架乃至每个版本的应用维护一个打好的包。而打包过程中，需要进行大量修改、配置、试错才能使本地应用运行环境和云端环境匹配。



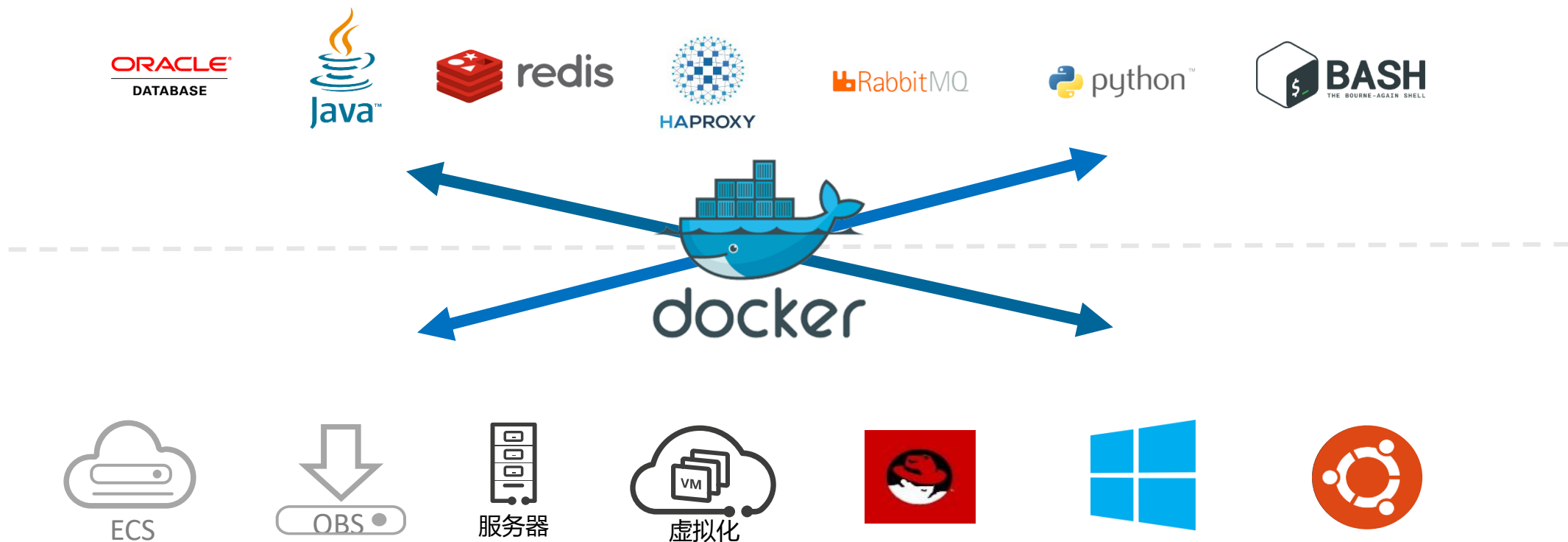
一键部署上云运行很痛快，打包过程很痛苦。

传统IT应用的挑战



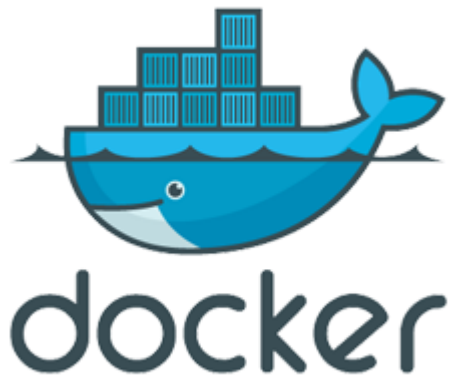
Docker项目的创新

- Docker镜像：
 - 容器镜像打包了应用及其依赖（包含完整操作系统的所有文件和目录）。
 - 容器镜像包含了应用运行所需要的所有依赖。只需在隔离的“沙盒”中运行该镜像，无需进行任何修改和配置即可运行应用。
 - 容器镜像核心在于实现应用及其运行环境整体打包以及打包格式统一。实现本地环境与云端环境的高一致性。



Docker项目介绍

- 2013年，dotCloud公司将Docker项目开源。
- Docker项目：
 - GitHub上开发的Moby开源项目的一部分。
 - 遵循Apache License 2.0许可证协议。
 - Go语言编写。
- Docker是一个开源的引擎，可以轻松的为任何应用创建一个轻量级的、可移植的、自给自足的容器。
- Docker公司目前推出两个版本：
 - Docker CE（社区版）
 - Docker EE（企业版）



旧logo



新logo

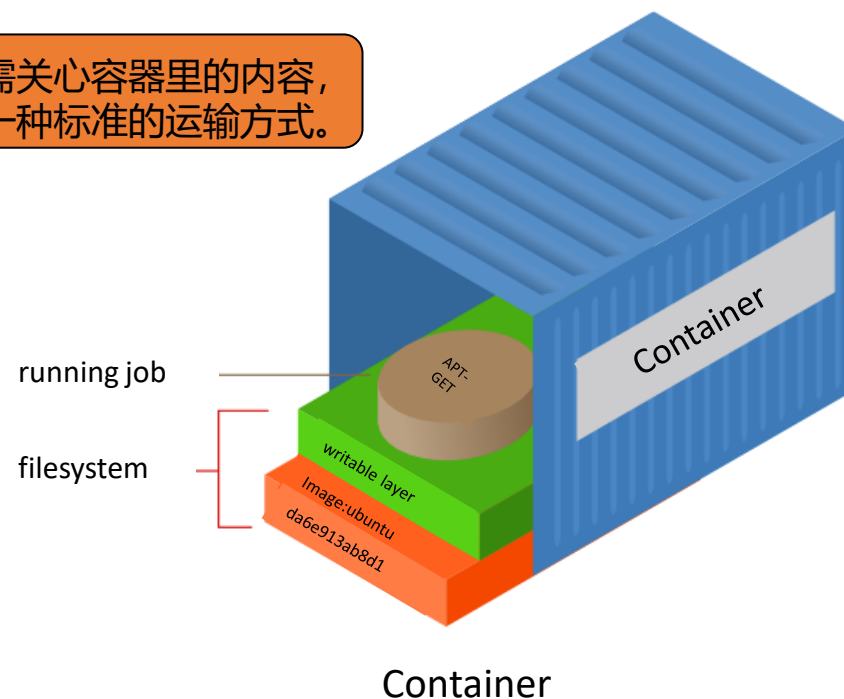
容器是什么

- 定义：容器是容器image运行时的实例。
- 通俗的理解：软件界的集装箱（隔离、封装）。

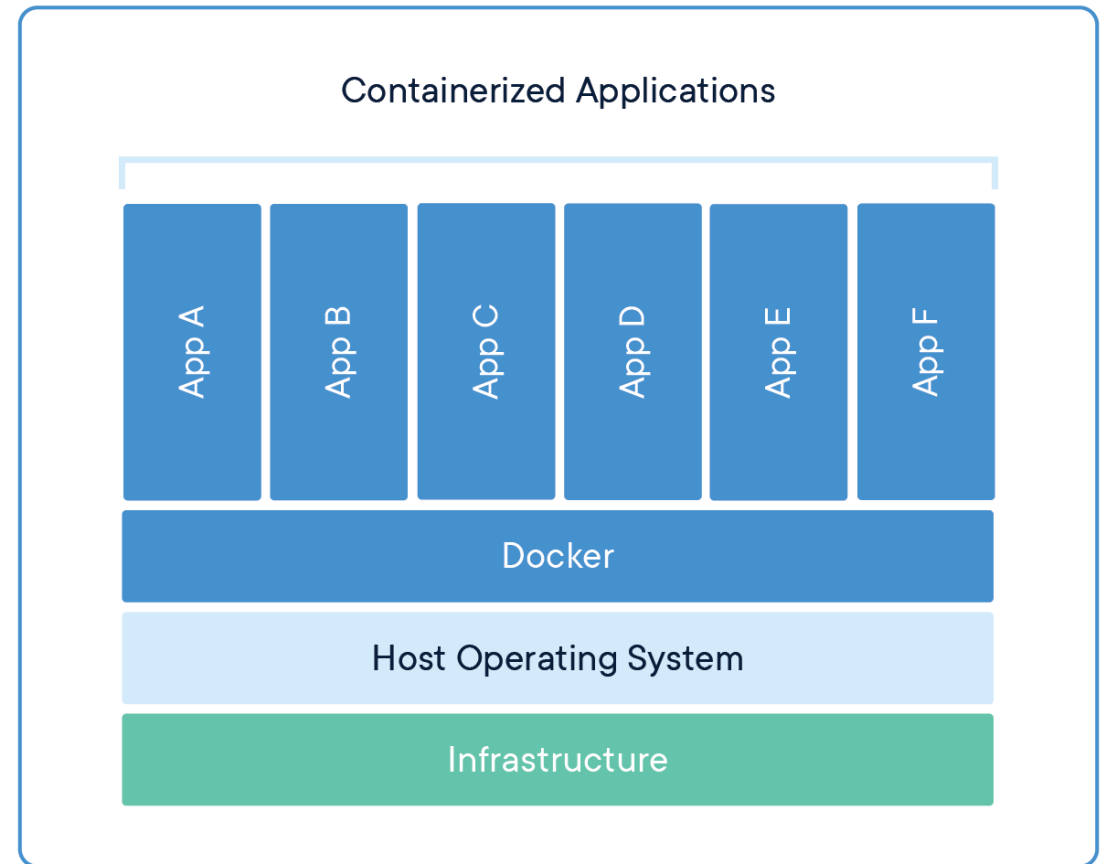
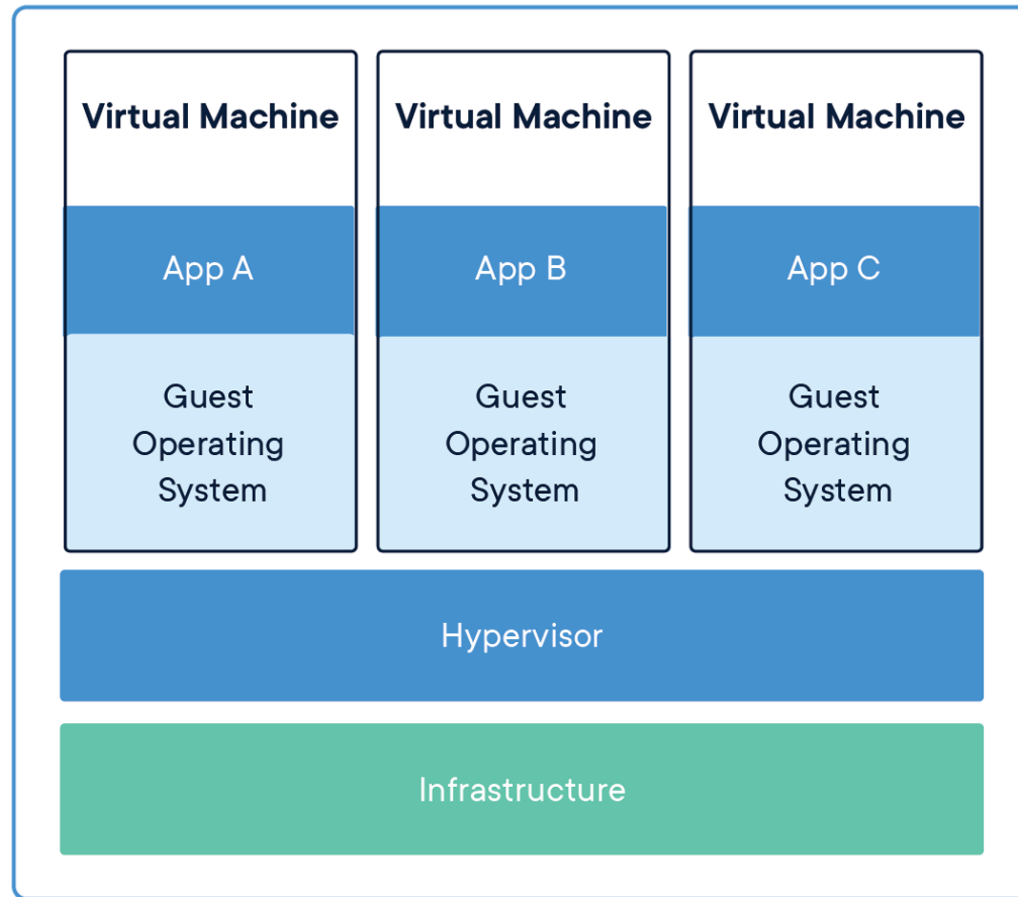


运输业

不需关心容器里的内容，
是一种标准的运输方式。



Container VS VM (1)



Container VS VM (2)

	容器	虚拟机
启动速度	秒甚至毫秒启动	数秒至数十秒
系统内核	共享内核	不共享内核
实现技术	利用Linux内核技术Namespace/Cgroup等实现。	依赖虚拟化技术实现，由Hypervisor层实现对资源的隔离
隔离效果	进程级别的隔离	系统资源级别的隔离
资源消耗 (性能)	容器中的应用只是宿主机上的一个普通进程	使用虚拟化技术，就会有额外的资源消耗和占用
资源调用 (敏捷性)	应用进程直接由宿主机OS管理	应用进程需经过Hypervisor的拦截和处理，才能调用系统资源
运行数量	一台服务器上能启动1000+容器	一台服务器上一般不超过100台虚拟机
应用	DevOps、微服务等	用于硬件资源划分
镜像	分层镜像	非分层镜像

CNCF基金会

- CNCF基金会：Cloud Native Computing Foundation
 - 隶属于Linux Foundation
 - 致力于云原生（Cloud Native）技术的普及和可持续发展。
- CNCF基金会下的明星项目
 - Kubernetes
 - Prometheus
 - Fluentd
 - CNI
 - gPRC

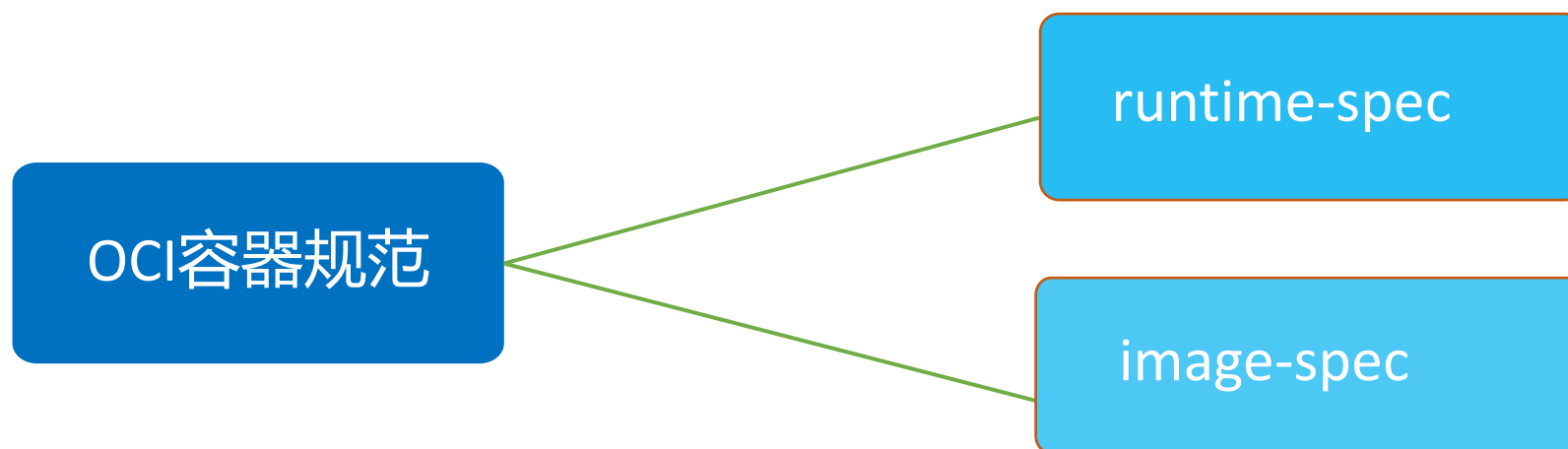


目录

1. 虚拟化技术
2. OpenStack
- 3. 容器技术**
 - 容器技术发展
 - 容器技术基础
 - 容器编排技术

OCI容器规范

- OCI: Open Container Initiative
 - 隶属于Linux Foundation
 - 旨在创建容器格式和容器runtime的开放行业标准



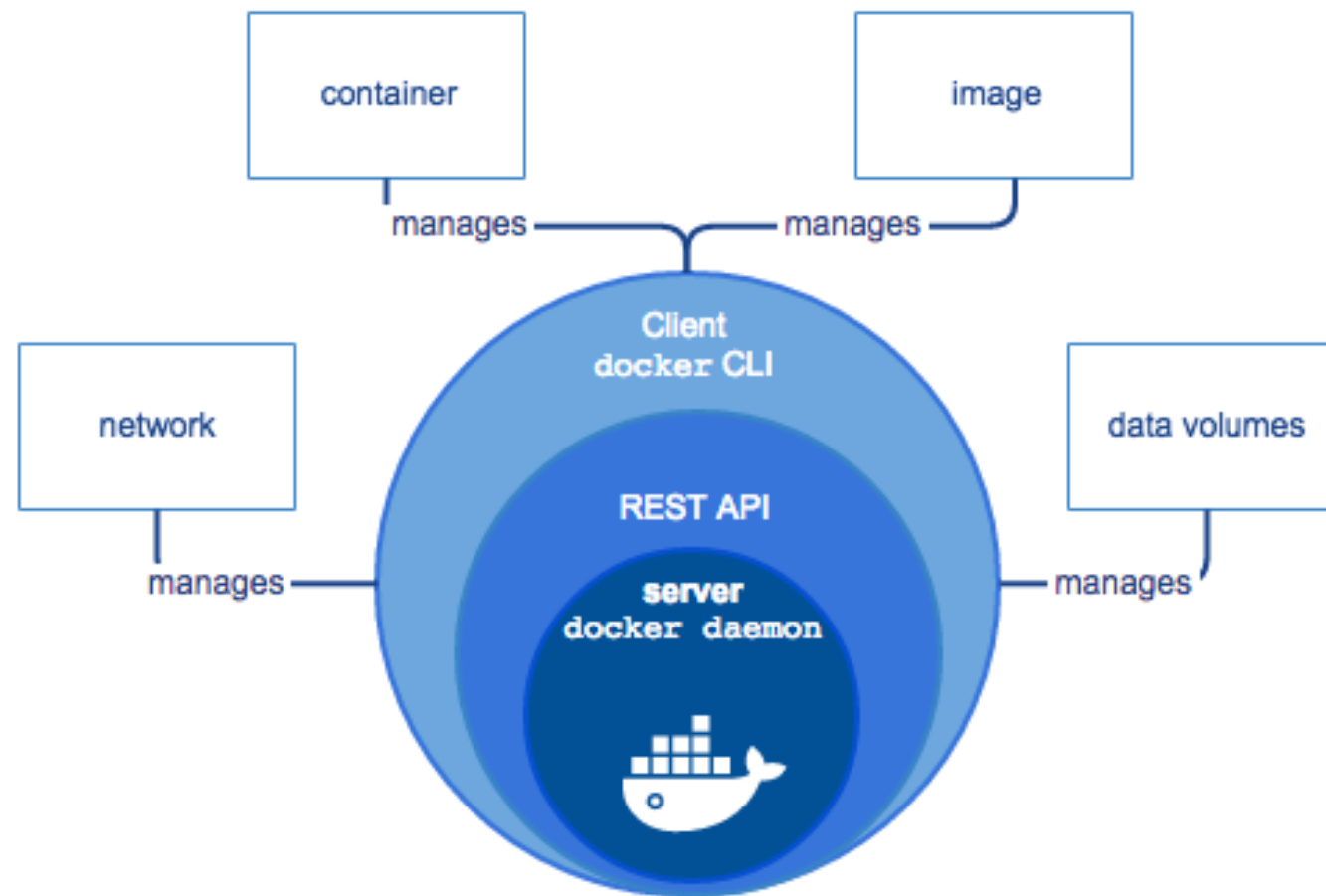
Runtime

- OCI下的Runtime规范旨在指定容器的配置、执行环境和生命周期。
- Runtime是对容器运行时的相关状态和操作进行管理的工具。Runtime定义了如下规范对容器进行管理：

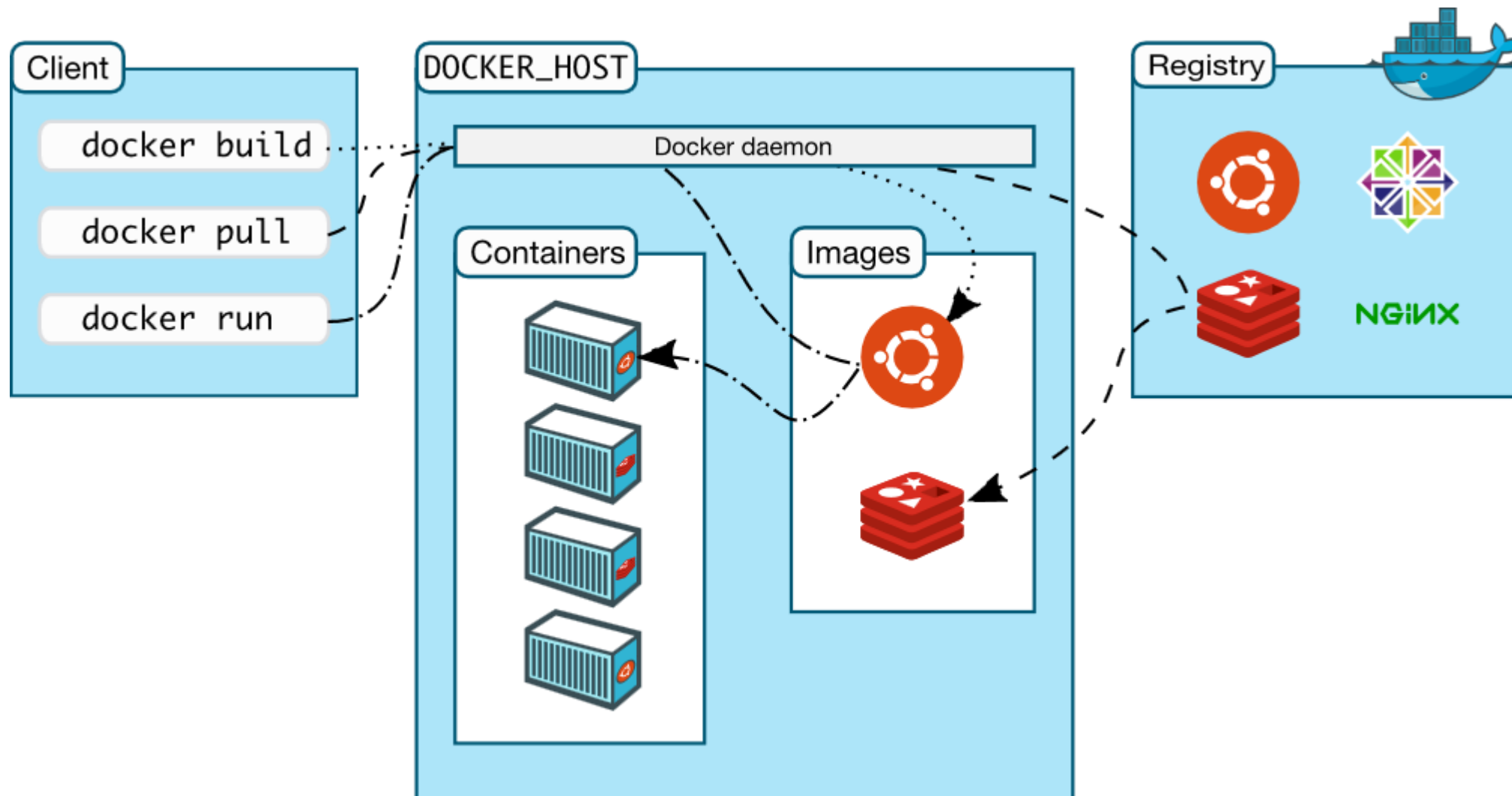
	内容
bundle.md (Filesystem Bundle)	bundle中包含了运行容器所需的所有信息，主要是config.json文件和rootfs。Runtime根据bundle启动容器。
config.md	包含对容器实施标准操作所必需的元数据，存放于config.json文件中。如oci版本、rootfs路径、mount目录、process、platform、容器hostname等。
config-linux.md	Linux平台上对config.md的扩展，内容也包含在config.json文件中。如namespace、devices、cgroupPath、resources、sysctl、readonlyPaths等。
runtime.md	定义了3部分内容：容器状态（如status、pid等）、容器相关操作（如create、kill等）、容器生命周期。
runtime-linux.md	是Linux平台上对runtime.md的补充。

- Runtime工具
 - runC是Docker公司2015年发布的符合OCI规范的runtime工具。
 - runC由Libcontainer演变而来。
 - 业界其他runtime工具有Linux上的LXC，CoreOS的rkt等。

Docker Engine介绍



Docker架构



查看docker服务状态

- 查看Docker engine状态

```
[root@localhost ~]# systemctl status docker.service
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; disabled; vendor preset: disabled)
   Active: active (running) since Tue 2019-07-16 08:52:39 EDT; 1 weeks 3 days ago
     Docs: https://docs.docker.com
  Main PID: 73043 (dockerd)
    Tasks: 35
   Memory: 299.6M
   CGroup: /system.slice/docker.service
           └─73043 /usr/bin/dockerd
              └─73049 docker-containerd --config /var/run/docker/containerd/containerd.toml
                 └─88328 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 8080 -container-ip 172.17.0.2 -container-port 80
                    └─88333 docker-containerd-shim -namespace moby -workdir /var/lib/docker/containerd/daemon/io.containerd.runtime.v1.lin
```

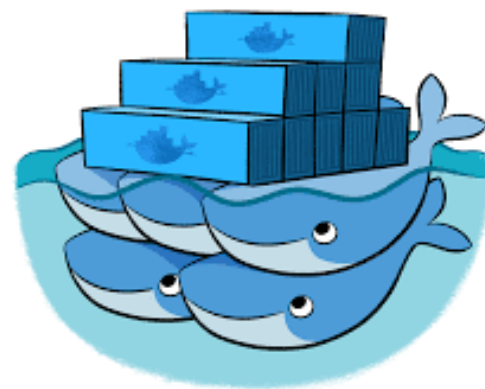
目录

1. 虚拟化技术
2. OpenStack
- 3. 容器技术**
 - 容器技术发展
 - 容器技术基础
 - 容器编排技术

什么是容器编排？

- 支持应用程序的容器的部署和组织称为**容器编排**，这是通过容器编排工具完成的。
- 典型的容器编排平台的基本特征包括：
 - 调度
 - 资源管理
 - 服务发现
 - 健康检查
 - 自动伸缩
 - 更新和升级

容器集群管理三巨头

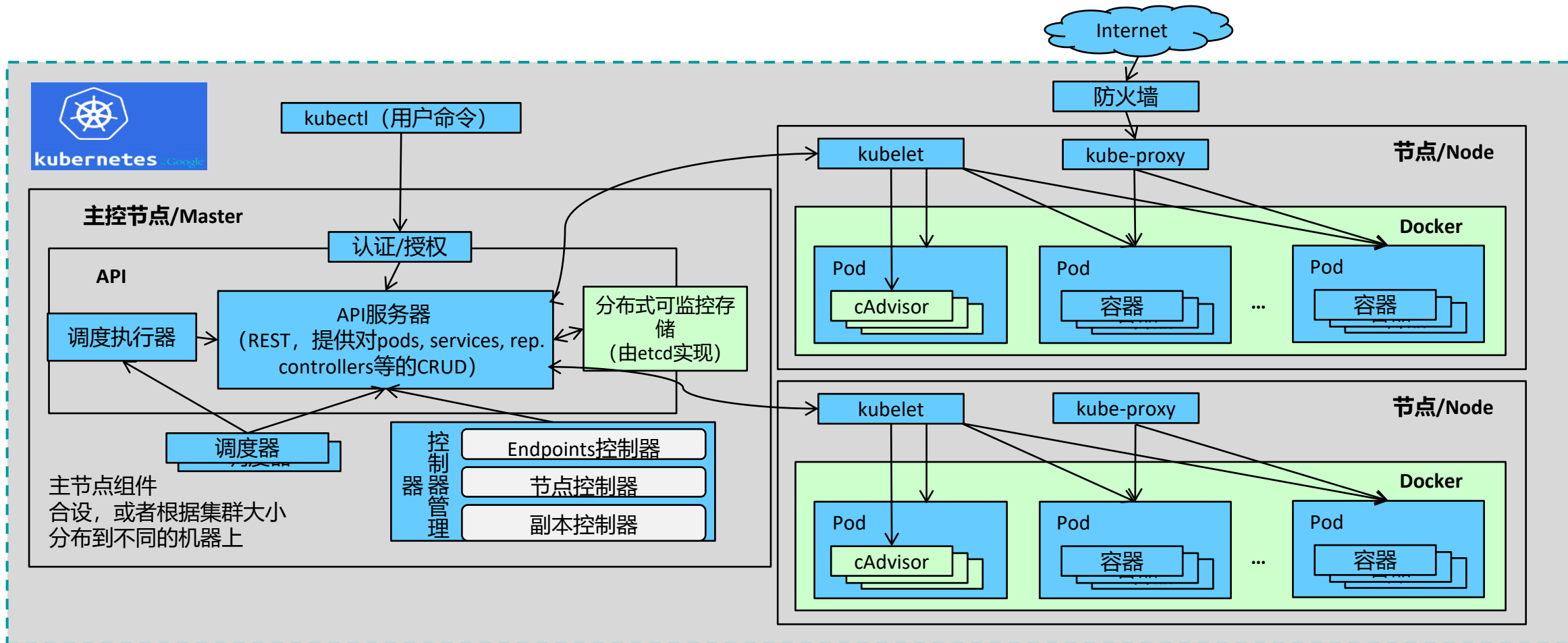


什么是Kubernetes?

- Kubernetes诞生于2014年，是Google根据15年大规模集群管理经验积累结合Docker技术向业界开源的容器编排管理技术；
- 推出后凭借其开放的开源生态，吸引了Redhat、Vmware、华为等业界顶级公司的参与，共同打造成了容器编排技术的事实标准；
- 2015年这些业界顶级公司成立了CNCF云原生基金会，成为云计算领域的顶级开源组织。



Kubernetes架构



Kubernetes的几个重要概念 (1)

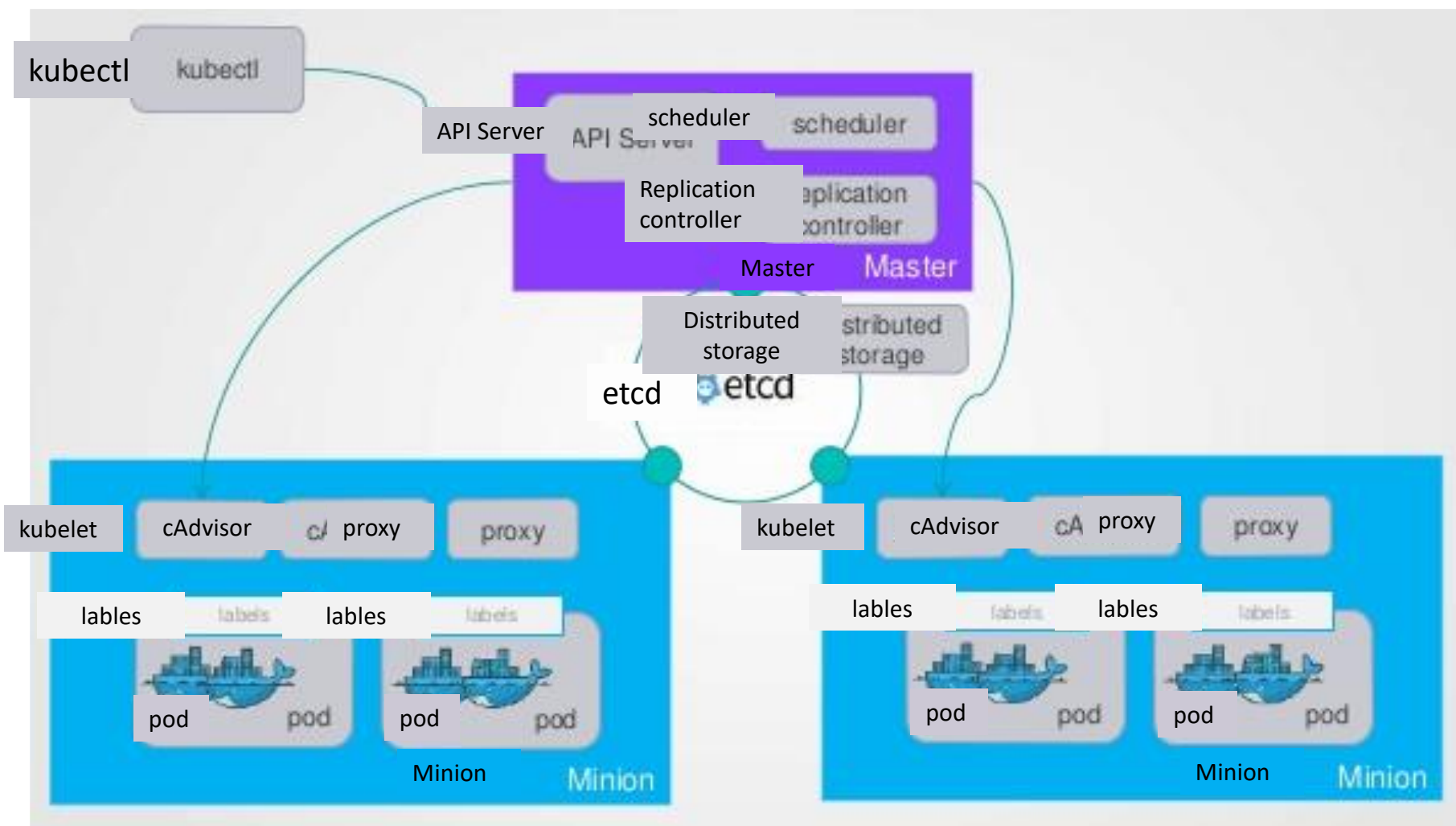
- Cluster :
 - Cluster 是计算、存储和网络资源的集合，Kubernetes 利用这些资源运行各种基于容器的应用。
- Master :
 - Master 是 Cluster 的大脑，它的主要职责是调度，即决定将应用放在哪里运行。Master 运行 Linux 操作系统，可以是物理机或者虚拟机。为了实现高可用，可以运行多个 Master。
- Node :
 - Node 的职责是运行容器应用。Node 由 Master 管理，Node 负责监控并汇报容器的状态，并根据 Master 的要求管理容器的生命周期。Node 运行在 Linux 操作系统，可以是物理机或者是虚拟机。
- Pod :
 - Pod 是 Kubernetes 的最小工作单元。每个 Pod 包含一个或多个容器。Pod 中的容器会作为一个整体被 Master 调度到一个 Node 上运行。

Kubernetes的几个重要概念 (2)

- Controller
 - Kubernetes 通常不会直接创建 Pod，而是通过 Controller 来管理 Pod 的。Controller 中定义了 Pod 的部署特性，比如有几个副本，在什么样的 Node 上运行等。为了满足不同的业务场景，Kubernetes 提供了多种 Controller，包括 Deployment、ReplicaSet、DaemonSet、StatefulSet、Job 等。
- Service
 - Kubernetes Service 定义了外界访问一组特定 Pod 的方式。Service 有自己的 IP 和端口，Service 为 Pod 提供了负载均衡。
 - Kubernetes 运行容器（Pod）与访问容器（Pod）这两项任务分别由 Controller 和 Service 执行。

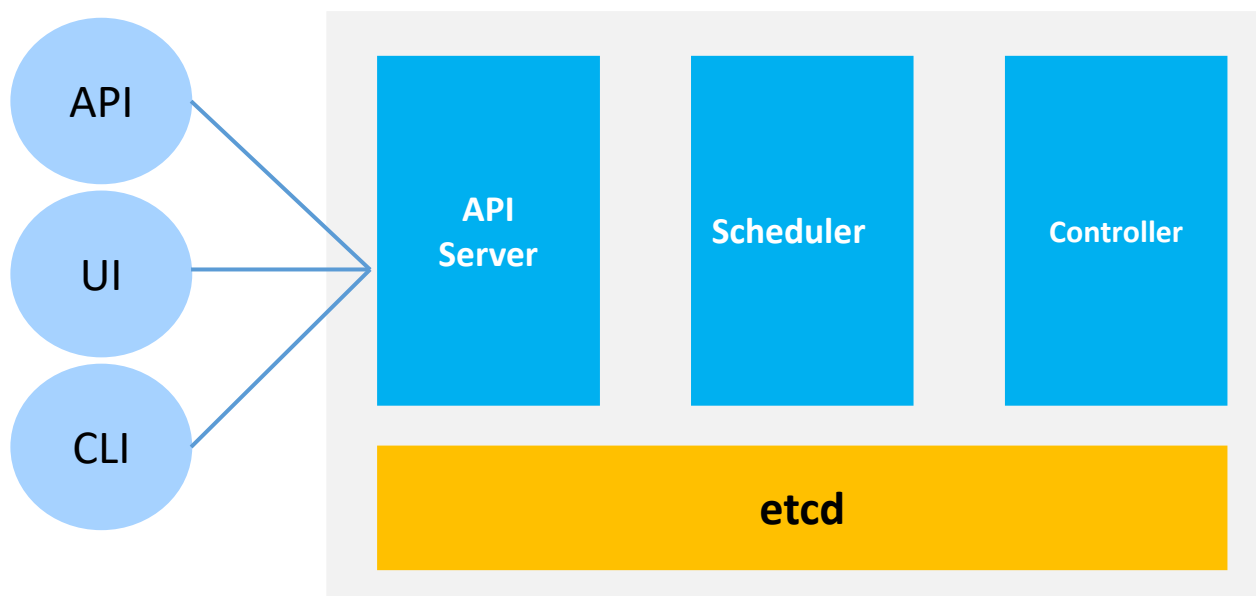
Kubernetes基本概念：Cluster

- 是计算、存储和网络资源的集合。
- 完整的、自治的、可正常运行工作负载的节点集合，包括管理节点和工作节点。

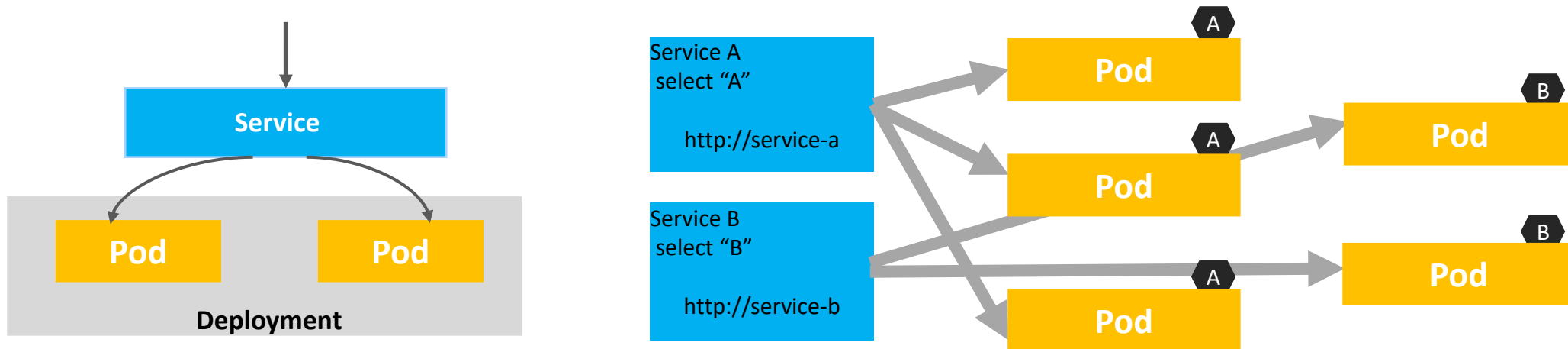


Kubernetes Master

- API server接收请求，如认证、鉴权等工作；
- Scheduler负责任务调度工作；
- Controller负责策略引擎，针对不同的工作负载执行不同的策略，如无状态应用，有状态应用等。
- etcd 负责保存 Kubernetes Cluster 的配置信息和各种资源的状态信息。



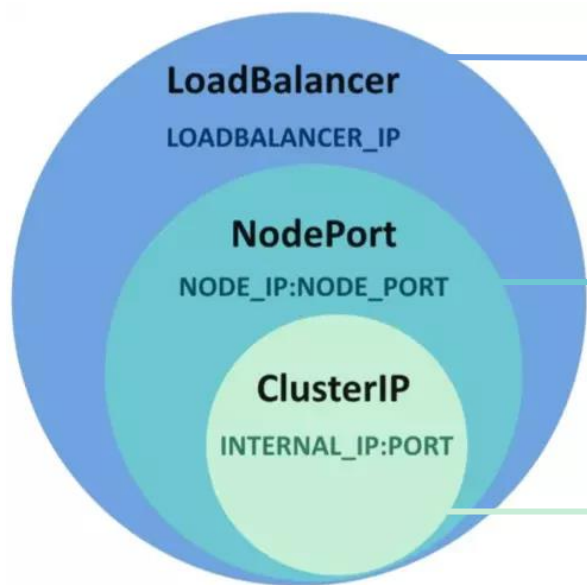
Kubernetes关键概念 - Service



- Service定义了pods的逻辑集合和访问这个集合的策略。Pods集合是通过定义Service时提供的Label选择器完成的；
- Service的引入旨在保证pod的动态变化对访问端透明，访问端只需要知道service的地址，由service来提供代理；
- Service的抽象使得前端客户和后端Pods进行了解耦；
- Kubernetes Services支持TCP和UDP协议，默认是TCP；
- Kubernetes支持两种基本的服务发现方法：环境变量和DNS。

Service的类型

- 支持多种访问方式实现
 - Cluster IP
 - NodePort
 - LoadBalancer

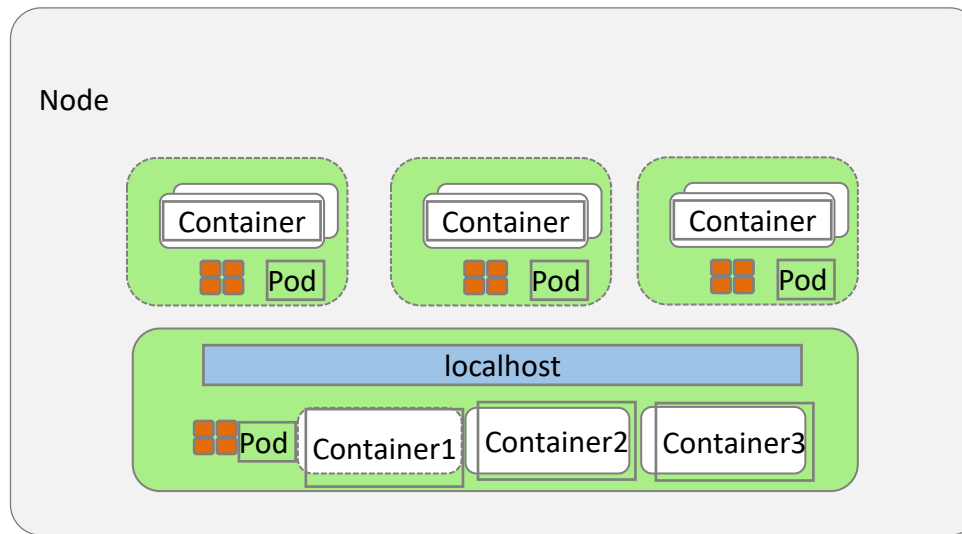


使用一个ClusterIP & NodePort, 但是会向cloud provider申请映射到service本身的负载均衡。

除了使用cluster ip外, 也将service的port映射到每个node的一个指定内部port上, 映射的每个node的内部port都一样

使用集群内的私有ip

Kubernetes基本概念 - POD (1)

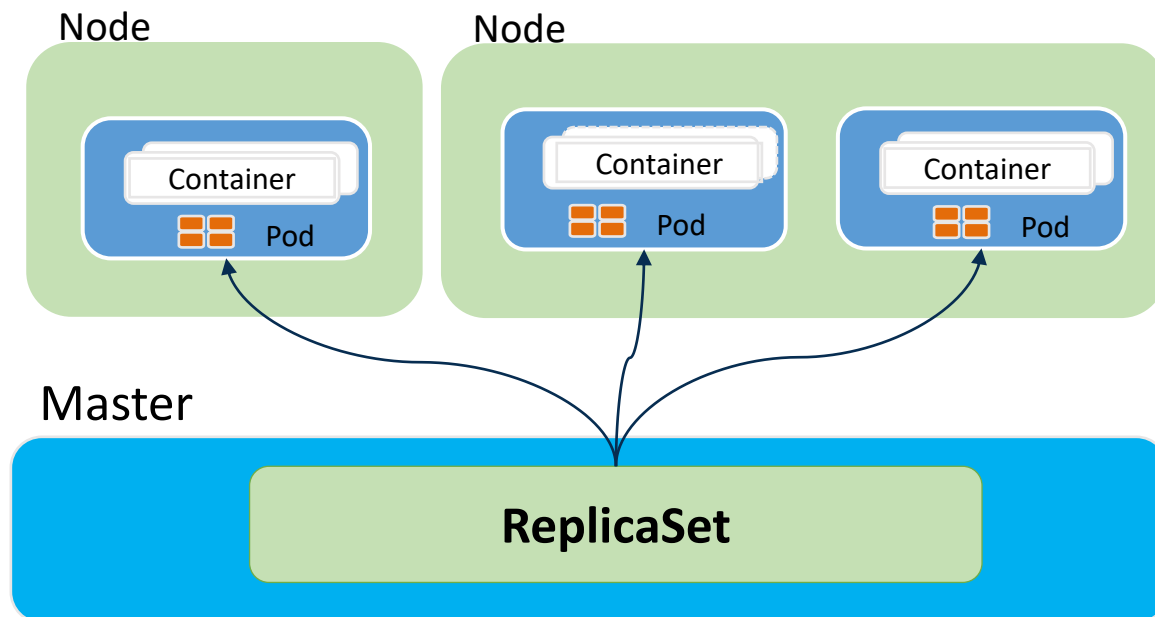


- 在Kubernetes中，pods是能够创建、调度、和管理的最小部署单元，是一组容器实例的集合
- 同一个Pod里的容器共享同一个网络命名空间，IP地址及端口空间和卷。
- 从生命周期来说，Pod是短暂的而不是长久的应用。Pods被调度到节点，保持在这个节点上直到被销毁。
- Pod的本质，实际上就是在扮演基础设施里“虚拟机”的角色；而容器，则是这个虚拟机里运行的用户程序。

Kubernetes 控制器 - ReplicaSet

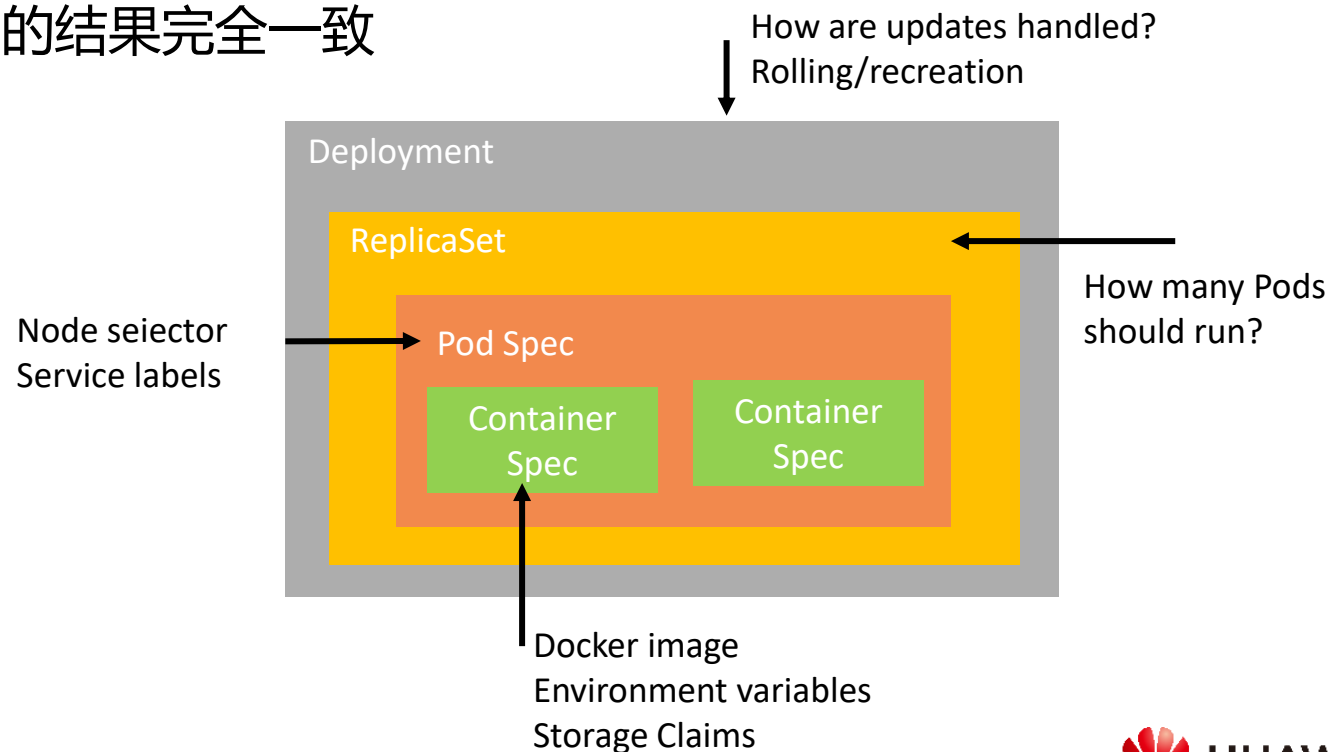
- ReplicaSet - 副本控制器

- 确保Pod的一定数量的副本(replica)在运行。如果超过这个数量，控制器会删除一些，如果少了，控制器会启动一些。
- ReplicaSet用于解决pod的扩容和缩容问题（作业副本和水平扩展）。
- 通常用于无状态应用



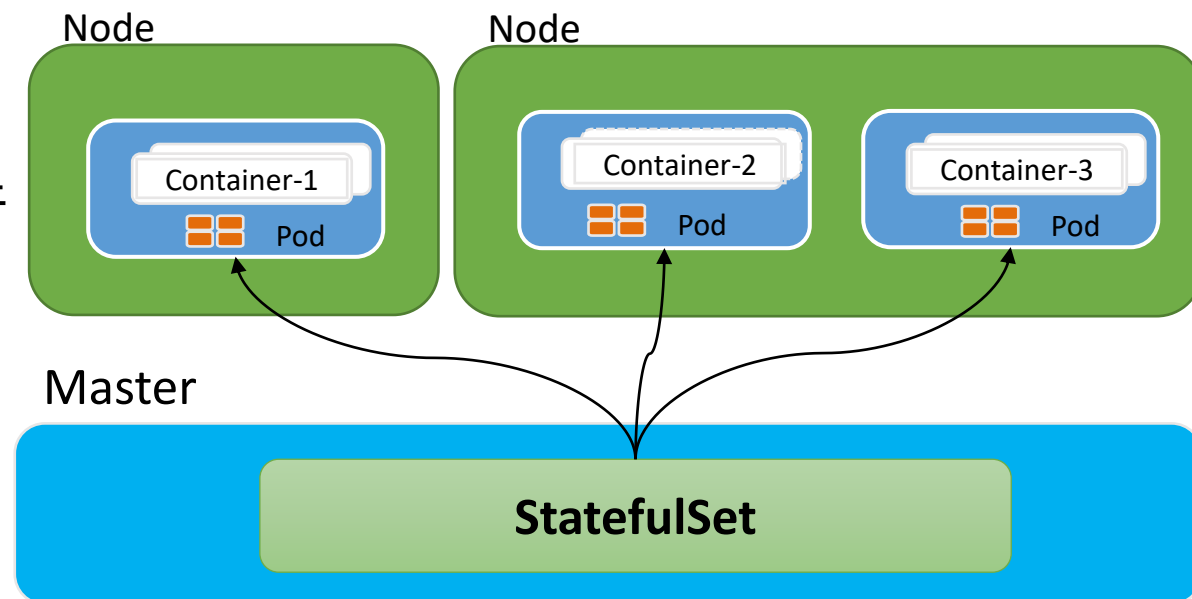
Kubernetes 控制器 - Deployment

- Deployment即在K8S中部署的无状态的应用：
 - 典型应用代表：Nginx, Tomcat
 - 不必为应用保持状态/持久化数据
 - 每个实例对于同一个请求响应的结果完全一致
 - 实例重启会丢失运行态数据



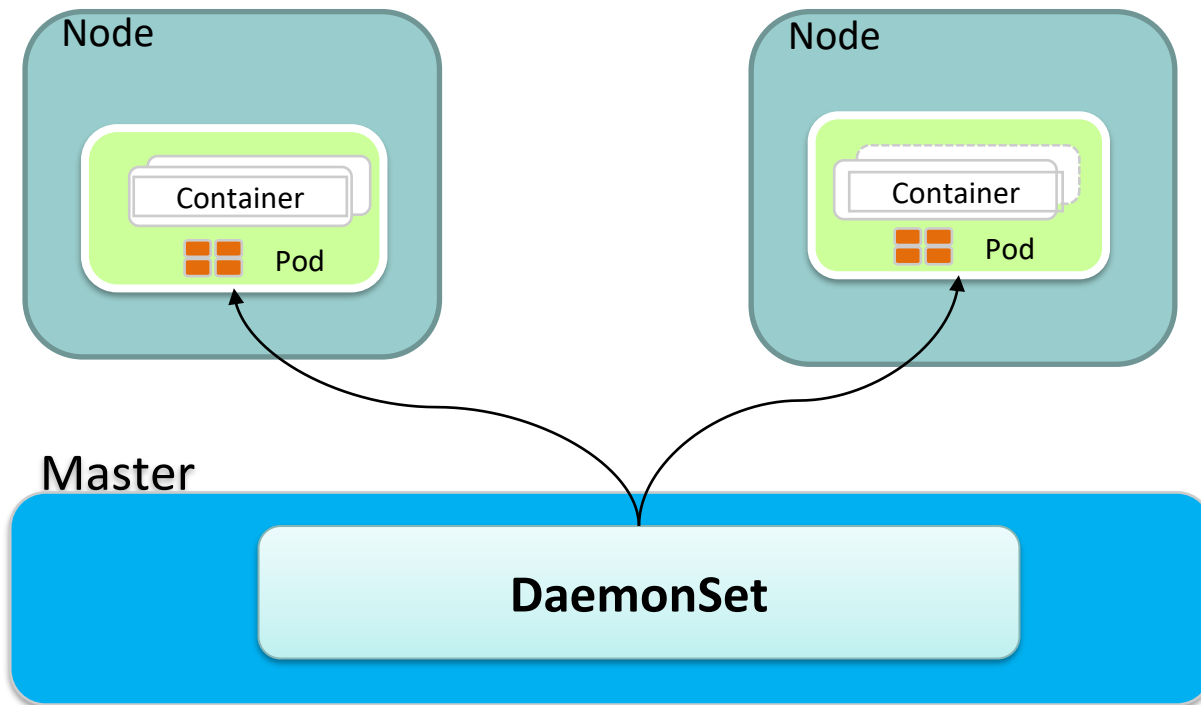
Kubernetes 控制器 - StatefulSet

- StatefulSet—有状态应用
 - 典型应用代表：mysql, redis
 - 每个实例具有唯一的网络标识符
 - 生命周期的操作将有序的在不同实例进行
 - 一般会挂载持久化存储保证数据持久性



Kubernetes 控制器 - DaemonSet

- DaemonSet能够让所有（或者一些特定）的Node节点运行同一个pod。当节点加入到kubernetes集群中，pod会被（DaemonSet）调度到该节点上运行，当节点从kubernetes集群中被移除，被（DaemonSet）调度的pod会被移除，如果删除DaemonSet，所有跟这个DaemonSet相关的pods都会被删除



Thank you.

把数字世界带入每个人、每个家庭、
每个组织，构建万物互联的智能世界。

Bring digital to every person, home, and
organization for a fully connected,
intelligent world.

**Copyright©2020 Huawei Technologies Co., Ltd.
All Rights Reserved.**

The information in this document may contain predictive statements including, without limitation, statements regarding the future financial and operating results, future product portfolio, new technology, etc. There are a number of factors that could cause actual results and developments to differ materially from those expressed or implied in the predictive statements. Therefore, such information is provided for reference purpose only and constitutes neither an offer nor an acceptance. Huawei may change the information at any time without notice.

